

รายงานโครงการ

ระบบทำนายผู้ป่วยไม่มาตามนัดของคลินิก

Clinic Appointment No-Show Prediction

รายวิชา CP413008 Machine Learning Engineering for Production
ภาคเรียนที่ 1 ปีการศึกษา 2569 มหาวิทยาลัยขอนแก่น

<https://github.com/KKU-Noshow-clinic/clinic-noshow-prediction>

รหัสนักศึกษา	ชื่อ-นามสกุล	Sec	หน้าที่ (Workstream)
673380532-1	นายศรัณยู เจริญผล	1	Lead / Framing / Integration
673380526-6	นายพิรพงษ์ ทองฤทธิ์	2	Data & Validation
673380078-7	นายธันว์ สว่างศรี	2	Features & Modeling
673380306-0	นายกิตติพันธ์ ไชไพบรวัน	1	Tracking, Registry & Pipeline
673380528-2	นายภูรินทร์ ศรีฐาน	1	Serving & Infra
673380075-3	นายทีฆานนท์ อัสวะภูมิ	2	Monitoring
673380313-3	นายณัฏทพงศ์ พยัคมะเริง	2	CI/CD & Testing

1. ภาพรวมโครงการ

1.1 ปัญหาและคุณค่าของระบบ

ผู้ป่วยที่ไม่มาตามนัด (no-show) ทำให้ช่องเวลาของแพทย์ว่างเปล่าโดยไม่เกิดประโยชน์ ผู้ป่วยรายอื่นที่ต้องการรักษาต้องรอนานขึ้น และคลินิกเสียรายได้ ระบบนี้ทำนายว่านัดหมายใดมีความเสี่ยงที่ผู้ป่วยจะไม่มา เพื่อให้คลินิกส่ง SMS หรือโทรเตือน หรือจัด overbooking ได้ทันเวลา

ทำไมใช้ Machine Learning แทนกฎธรรมดา: การไม่มาตามนัดเกิดจากหลายปัจจัยร่วมกัน (ระยะเวลาจองล่วงหน้า อายุ ประวัติการไม่มา การได้รับ SMS วันในสัปดาห์ ยานที่อยู่) ความสัมพันธ์ไม่เป็นเส้นตรงและเปลี่ยนแปลงตามเวลา การเขียนกฎด้วยมือจึงครอบคลุมได้ยากกว่าการให้โมเดลเรียนรู้จากข้อมูลจริง

1.2 ชุดข้อมูล

ใช้ชุดข้อมูล Medical Appointment No Shows จาก Kaggle (joniarroba/noshowappointments) มีนัดหมาย 110,527 รายการ 14 คอลัมน์ จากประเทศบราซิล ช่วงเดือนเมษายนถึงมิถุนายน 2016 Target คือ No-show == "Yes" (ผู้ป่วยไม่มา) เป็นคลาส 1 ซึ่งมีประมาณ 20% ของข้อมูล

ชุดข้อมูลมีขนาดเกินเกณฑ์ขั้นต่ำ 5,000 ตัวอย่าง มีเวลานัดหมายจึงแบ่งชุดข้อมูลตามเวลาได้ และจำลอง data drift กับ concept drift ได้

1.3 เครื่องมือและเหตุผลที่เลือก

หน้าที่	เครื่องมือ	เหตุผล
Version Control	Git + GitHub	มี Pull Request สำหรับ code review, GitHub Actions สำหรับ CI และ Issues สำหรับแบ่งงานในที่เดียว ทีมคุ้นเคยและใช้ GitHub Desktop ได้ (GitLab เต้นเรื่อง self-host แต่โปรเจกต์ใช้ข้อมูลสาธารณะจึงไม่จำเป็น)
Containerization	Docker + Docker Compose	ล็อกเวอร์ชัน Python และบริการทุกตัว รันได้เหมือนกันทุกเครื่อง และเปิดบริการ 5 ตัว (API, MLflow, Prefect, Prometheus, Grafana) ด้วยคำสั่งเดียว Kubernetes ซับซ้อนเกินไปสำหรับระบบเครื่องเดียว
Data Validation	Pandera	ตรวจสอบ pandas DataFrame ได้โดยตรง schema เป็นโค้ด Python ที่อ่านง่ายและเก็บใน Git ได้ เบากว่า Great Expectations และไม่ต้องใช้ TensorFlow แบบ TFDV (ซึ่งติดตั้งบน Windows ไม่ได้)
Experiment Tracking	MLflow	บันทึกพารามิเตอร์ ตัวชี้วัด เวอร์ชันโค้ดและข้อมูล ไฟล์โมเดล และสภาพแวดล้อมที่ใช้รัน เพื่อเปรียบเทียบและทำการทดลองซ้ำ
Model Registry	MLflow Model Registry	จัดการเวอร์ชันโมเดล โดยใช้ alias challenger, champion และ previous_champion ชี้ไปยังเวอร์ชันที่เกี่ยวข้อง พร้อมเชื่อมขั้นตอนตรวจสอบก่อนใช้งาน (gate) เลื่อนเป็นโมเดลหลัก (promote) และย้อนกลับเวอร์ชันเดิม (rollback)
Pipeline Orchestration	Prefect	ควบคุมลำดับงานเป็น flow (DAG) ติดตามสถานะแต่ละขั้น และเรียก failure hook แจ้งเตือนเมื่อทำงานล้มเหลว
Model Serving	FastAPI + Uvicorn	เขียน API ด้วย Python เดียวกับ pipeline ตรวจสอบ input ด้วย Pydantic และสร้างหน้าเอกสาร /docs อัตโนมัติ

หน้าที่	เครื่องมือ	เหตุผล
Monitoring	Prometheus + Grafana (+ Evidently สำหรับ drift)	Prometheus เก็บ metrics ของ API จาก <code>/metrics</code> และ Grafana แสดง dashboard
CI/CD	GitHub Actions	รันอัตโนมัติทุก Pull Request ตรวจ 3 ด้าน (โค้ด ข้อมูล โมเดล) ใช้ action สำเร็จรูป เช่น <code>setup-uv</code>

2. การวางกรอบปัญหาและการวางแผน (ส่วนที่ 1)

2.1 AI Project Canvas

AI Project Canvas — Clinic Appointment No-Show Prediction

1. Value Proposition

ปัญหา: ผู้ป่วยไม่มาตามนัดประมาณ 20% ทำให้คลินิกเสียสล็อตแพทย์ เสียรายได้ และผู้ป่วยคนอื่นต้องรอคิวนานขึ้น

คุณค่า: ทำนายล่วงหน้าว่านัดไหนเสี่ยงที่ผู้ป่วยจะไม่มา เพื่อให้คลินิกโทรหรือส่ง SMS เตือนเฉพาะกลุ่มเสี่ยง หรือจัด overbooking ได้ทัน จึงลดสล็อตที่ว่างเปล่าและลดต้นทุนการเตือนแบบเหวี่ยงแห

2. Data

ข้อมูลที่ใช้: ประวัตินัดหมาย ได้แก่ วันจอง, วันนัด, อายุ, เพศ, ยาน, โรคประจำตัว, สวัสดิการ (Scholarship), การได้รับ SMS และผลว่ามาหรือไม่มา (label)

แหล่งข้อมูล: Kaggle Medical Appointment No Shows (บราซิล, 29 เม.ย. – 8 มิ.ย. 2016)

ปริมาณ: 110,527 นัด 14 คอลัมน์ ไฟล์ CSV เหลือ 110,521 แถวหลัง clean

การควบคุม: ตรวจสอบ schema ด้วย Pandera และใช้ SHA-256 เป็น data version

3. Skills

Data Engineer: ingest, cleaning, validation

ML Engineer: feature engineering, LightGBM, จัดการ class imbalance, SHAP

MLOps Engineer: MLflow, Prefect, Docker, CI/CD และ monitoring

Backend Developer: FastAPI, load testing

ผู้เชี่ยวชาญงานคลินิก/เวชระเบียน: ให้คำปรึกษาว่าปัจจัยใดมีผลต่อการมาตามนัด และตรวจสอบว่านำไปใช้ได้จริง

4. Output

ตัวชี้วัดหลัก (Optimizing): No-show Recall ≥ 0.60 ยิ่งสูงยิ่งดี เพราะการพลาดผู้ที่จะไม่มา (FN) แพงกว่าการเตือนเกิน (FP) โมเดลที่เลือกได้ val recall 0.80 และ test recall 0.755

ตัวชี้วัดเสริม (Satisficing):

Precision ≥ 0.25 (ติดตามเพิ่ม)

PR-AUC ≥ 0.299 (baseline)

p95 latency ≤ 200 ms (วัดได้จริง 80 ms)

error rate $\leq 1\%$

ขนาดโมเดล ≤ 100 MB

5. Integration

API (FastAPI ใน Docker) ให้ระบบนัดหมายของคลินิกเรียกใช้

แบบ batch (/predict_batch): รันคืนก่อนวันนัด แล้วส่งรายชื่อผู้ที่เสี่ยงไปให้ระบบส่ง SMS หรือให้เจ้าหน้าที่โทรเตือน

แบบ real-time (/predict): แสดงคะแนนความเสี่ยงทันทีตอนเจ้าหน้าที่จองนัด

โมเดลมาจาก MLflow Registry (alias champion) ทั้งระบบ retrain, gate และ deploy อัตโนมัติผ่าน Prefect

6. Customers

กลุ่มหลัก: เจ้าหน้าที่เวชระเบียนหรือฝ่ายนัดหมายของคลินิกและโรงพยาบาล

กลุ่มรอง: ผู้บริหารคลินิก (วางแผนสล็อตและ overbooking)

ผู้ได้รับประโยชน์ทางอ้อม: ผู้ป่วย ได้รับการเตือนและได้คิวเร็วขึ้น

7. Stakeholders

ภายใน: แพทย์และพยาบาล, ฝ่ายเวชระเบียน, ฝ่าย IT, ฝ่ายการเงิน, ผู้บริหาร

ภายนอก: ผู้ป่วย, ผู้ให้บริการ SMS, หน่วยงานกำกับดูแลข้อมูลสุขภาพ (PDPA)

8. Cost

บุคลากร: ทีมพัฒนา 7 คน, ผู้ดูแลระบบหลัง deploy

โครงสร้างพื้นฐาน: server สำหรับ API, MLflow, Prefect, Prometheus/Grafana (ใช้เครื่องมือ open-source ทั้งหมด จึงไม่มีค่า license)

ข้อมูล: ไม่ต้องจ้าง label เพราะ label (มาหรือไม่มา) เกิดขึ้นเองหลังวันนัด มีแค่การจัดเก็บข้อมูลและค่าดูแลเรื่อง PDPA

ต้นทุนดำเนินงาน: ค่า SMS หรือเวลาโทรเตือนกลุ่มเสี่ยง (เดือนประมาณ 3.5 คนต่อ no-show จริง 1 ราย)

9. Revenue

ลดการสูญเสีย: สล็อตที่เคยว่างเพราะผู้ป่วยไม่มา กลับมาสร้างรายได้ได้ ด้วยการเตือนให้มา หรือให้ยกเลิกล่วงหน้าแล้วนำสล็อตไปให้ผู้ป่วยรายอื่น

ลดต้นทุน: เตือนเฉพาะกลุ่มเสี่ยงแทนการเตือนทุกคน ช่วยลดค่า SMS และเวลาเจ้าหน้าที่

รายได้ใหม่ (ถ้าต่อยอด): เปิดบริการ API ทำนาย no-show ให้คลินิกอื่นแบบ subscription

2.2 Optimizing metric และ Gating metric

ข้อมูลไม่สมดุล (no-show 20%) โมเดลที่ทายว่าทุกคนมาตามนัดจะได้ accuracy ประมาณ 80% โดยจับผู้ป่วยที่ไม่มาไม่ได้เลย ทีมจึง **ไม่ใช่ accuracy** แต่ใช้ตัวชี้วัดที่เน้นคลาส no-show

ประเภท	ตัวชี้วัด	เกณฑ์ (<code>configs/slo.yaml</code>)	เหตุผล
Optimizing	No-show recall (ที่ threshold ที่เลือก) และ PR-AUC	ยิ่งสูงยิ่งดี	เป้าหมายคือจับผู้ป่วยที่จะไม่มาให้ได้มากที่สุดเพื่อส่งเตือน
Gating	Validation PR-AUC	≥ 0.298904 (= ผลของ baseline)	โมเดลใหม่ต้องไม่แย่กว่า baseline
Gating	Validation no-show recall	≥ 0.60	ต้องจับผู้ป่วยไม่มาได้อย่างน้อย 60%
Gating	ขนาดไฟล์โมเดล	≤ 100 MB	โหลดเร็วและใช้หน่วยความจำไม่มาก
Gating	เทียบกับโมเดล production	<code>must_beat_production : true</code>	challenger ต้องดีกว่า champion จึง promote
Gating (ระบบ)	Latency p50 / p95	≤ 50 ms / ≤ 200 ms	ตอบทันเวลาเมื่อเจ้าหน้าที่ใช้งาน

2.3 การวัดความสำเร็จเชิงธุรกิจ

ผลของโมเดลที่เลือกบนชุด test: recall 0.7548 และ precision 0.2841 แปลว่าจากผู้ป่วยที่ไม่มาจริง 100 คน ระบบเตือนได้ประมาณ 75 คน และจากผู้ป่วย 100 คนที่ระบบเลือกเตือน มีประมาณ 28 คนที่ไม่มาจริง ทีมยอมให้ precision ต่ำเพื่อแลกกับ recall สูง เพราะต้นทุน SMS ต่ำมากเมื่อเทียบกับชองนัดแพทย์ที่เสียไป

ตัวชี้วัดธุรกิจ	ตัวชี้วัดโมเดล (configs/slo.yaml)	เหตุผล
อัตรา no-show ผู้ที่ได้รับการเตือนทัน	Recall คลาส no-show (optimizing metric, gate ≥ 0.60)	recall = สัดส่วนผู้ที่จะไม่มาที่ระบบจับได้
ค่า SMS/เวลาโทรต่อ no-show 1 รายที่จับได้	Precision (gate ≥ 0.25 ที่เสนอ)	precision 0.28 → เตือน ~3.5 คนต่อ no-show จริง 1 ราย
อัตราการใช้สล็อตแพทย์	Recall $\times$ อัตราที่การเตือนทำให้ผู้ป่วยมา/ยกเลิกล่วงหน้า	ยกเลิกล่วงหน้าทำให้นาสล็อตไปให้คนอื่นได้
คุณภาพการจัดอันดับความเสี่ยงโดยรวม	PR-AUC (gating $\geq$ baseline 0.299)	ไม่ขึ้นกับ threshold และเหมาะกับข้อมูลไม่สมดุล

ตัวอย่างจากชุด test : ชุด test มี 26,450 นัด เป็น no-show ประมาณ 4,890 นัด ระบบเตือนประมาณ 13,000 นัด (ราวครึ่งหนึ่งของนัดทั้งหมด) และจับผู้ที่จะไม่มาได้ประมาณ 3,690 นัด (75%) เมื่อเทียบกับการเตือนผู้ป่วยทุกคน ระบบลดปริมาณการเตือนลงประมาณครึ่งหนึ่ง แต่ยังครอบคลุมผู้ที่จะไม่มาได้ 3 ใน 4

ตัวอย่างความคุ้มค่า (ค่าสมมติ) : จากชุด test ระบบส่งการเตือนล่วงหน้าถึงผู้ป่วยกลุ่มเสี่ยงได้ประมาณ 3,690 ราย หรือ 75% ของผู้ที่จะไม่มาตามนัด (recall 0.755) ผู้ป่วยกลุ่มนี้จึงได้รับการเตือนทันเวลา ทำให้มีโอกาสมาตามนัด แจ้งเลื่อนนัด หรือยกเลิกนัด แทนที่จะหายไปเฉย ๆ สมมติว่าการเตือนช่วยให้ผู้ป่วยประมาณ 10% ของกลุ่มมาตามนัดหรือเลื่อนนัด จะมีผู้ป่วยประมาณ 369 รายที่ได้รับการรักษาตามนัด และคลินิกสามารถนำสล็อตที่ถูกเลื่อนไปจัดให้ผู้ป่วยรายอื่นที่รอคิวอยู่ได้ ทำให้ผู้ป่วยโดยรวมได้รับการรักษาเร็วขึ้น

ด้านต้นทุน : การเตือน SMS ประมาณ 13,000 ข้อความ (ผู้ป่วยที่จับได้ 3,690 ราย $\div$ precision 0.28 หมายถึงต้องเตือนประมาณ 3.5 คนจึงจะเจอผู้ที่ไม่มาจริง 1 คน) ถ้าข้อความละ 0.5 บาท จะได้ค่าใช้จ่ายราว 6,500 บาท หรือเฉลี่ยประมาณ 18 บาทต่อผู้ป่วย 1 รายที่มาตามนัดหรือเลื่อนนัดทัน ซึ่งถือว่าต่ำมากเมื่อเทียบกับผลเสียจากการพลาดนัด ซึ่งทำให้เสียเวลาแพทย์หรือผู้ป่วยคนอื่น ตัวเลขนี้จึงสนับสนุนการเลือก recall สูง เพราะการเตือนผู้ป่วยกลุ่มเสี่ยงให้ได้มากที่สุดสำคัญกว่าการเสียค่า SMS ให้ผู้ที่ไม่มาตามนัดอยู่แล้ว

(ค่าทั้งหมดเป็นค่าสมมติเพื่อแสดงวิธีคิด ต้องใช้ข้อมูลจริงของคลินิกเมื่อนำไปใช้)

วิธีวัดผล : ใช้ A/B test โดยสุ่มแบ่งผู้ป่วยในช่วงทดลอง 4 สัปดาห์ออกเป็น 2 กลุ่ม กลุ่ม A ใช้วิธีเดิมของคลินิก ส่วนกลุ่ม B เตือนเฉพาะนัดที่ระบบ flag ว่าเสี่ยง จากนั้นเปรียบเทียบอัตรา no-show และต้นทุนการเตือนด้วยการทดสอบทางสถิติ ถือว่าสำเร็จเมื่อกลุ่ม B มีอัตรา no-show ต่ำกว่า หรือมีอัตราเท่าเดิมแต่ต้นทุนต่ำกว่า เนื่องจาก label ทราบผลหลังวันนัด ทีมจึงคำนวณ recall และ precision จริงย้อนหลังได้ทุกสัปดาห์ และใช้ข้อมูลเดียวกันนี้ตรวจจับ concept drift ในหัวข้อ 7.1 ได้ด้วย

ข้อจำกัด : โครงการนี้ใช้ข้อมูลย้อนหลัง จึงยังไม่ได้ทำ A/B test จริง ตัวเลขข้างต้นเป็นการประมาณจากชุด test

3. ข้อมูลและการตรวจสอบคุณภาพ (ส่วนที่ 2)

งานส่วนนี้: Data & Validation (นายพิรพงษ์ ทองฤทธิ์)

3.1 การนำเข้าข้อมูลและควบคุมเวอร์ชัน

ดาวน์โหลดข้อมูลด้วย `scripts/download_data.py` ซึ่งคำนวณ SHA-256 ของไฟล์แล้วเทียบกับค่าใน `configs/params.yaml` ถ้าไม่ตรงสคริปต์จะหยุดและแจ้ง `SHA-256 mismatch` ทำให้ทุกคนในทีมใช้ข้อมูลชุดเดียวกัน ค่า SHA-256 นี้ใช้เป็นเวอร์ชันของข้อมูลและถูกบันทึกลง MLflow ทุกครั้งที่เทรน ส่วนไฟล์ข้อมูลไม่ได้เก็บใน Git (กันไว้ด้วย `.gitignore`)

```
● (clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction> uv run python scripts/download_data.py
data\raw\noshow.csv sha256=9132d3e7d0246617df9041d3764f20ad6f08e7b0d9f0997fa254fc5e52eda27d
○ (clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction> |
```

ภาพที่ 1 ดาวน์โหลดข้อมูลแล้ว SHA-256 ตรงกับเวอร์ชันที่กำหนด

```
configs > ! params.yaml
4 data:
5 # Kaggle: joniarroba/noshowappointments (110,527 appointments, Brazil, Apr-Jun 2016)
6 kaggle_dataset: joniarroba/noshowappointments
7 kaggle_file: KaggleV2-May-2016.csv
8 sha256: "9132d3e7d0246617df9041d3764f20ad6f08e7b0d9f0997fa254fc5e52eda27d" # data version v1
9 raw_path: data/raw/noshow.csv
10 processed_dir: data/processed
```

ภาพที่ 2 ค่า SHA-256 (data version v1) ใน `configs/params.yaml`

อ้างอิง: [scripts/download_data.py](#)

3.2 การสำรวจข้อมูล (EDA)

สคริปต์ `scripts/eda.py` สรุปข้อมูล 7 หัวข้อ ผลอยู่ใน `docs/eda.txt` สิ่งที่น่าสนใจ:

หัวข้อ	ผล
ขนาด	110,527 แถว × 14 คอลัมน์
สัดส่วน No-show	Yes 22,319 (20.19%) / No 88,208 (79.81%) ข้อมูลไม่สมดุล
ช่วงเวลา	วันนัด 29 เม.ย. – 8 มิ.ย. 2016 (27 วันที่มีนัด) วันจองย้อนไปถึง พ.ย. 2015
ค่าว่าง / ซ้ำ	ค่าว่าง 0, แถวซ้ำ 0, AppointmentID ซ้ำ 0
ผู้ป่วยนัดหลายครั้ง	ผู้ป่วยไม่ซ้ำ 62,299 คน, 24,379 คน (39%) นัดมากกว่า 1 ครั้ง, สูงสุด 88 ครั้งต่อคน
ค่าผิดปกติ	Age < 0 = 1 แถว, Age > 100 = 7 แถว, Handcap 2–4 = 199 แถว, จองหลังวันนัด = 5 แถว, PatientId ทศนิยม 5 ค่า

ข้อสังเกตเรื่องเวลา: ไม่มีนัดวันเสาร์-อาทิตย์ (ยกเว้น 14 พ.ค. มี 39 นัด) และไม่มีข้อมูลวันที่ 21–23 กับ 26–29 พ.ค. (รวมวันหยุด Corpus Christi ของบราซิล) อัตรา no-show รายวันในเดือนพฤษภาคมอยู่ที่ประมาณ 20–23% และลดลงเหลือประมาณ 17–19% ในเดือนมิถุนายน

อ้างอิง: [docs/eda.txt](#)

3.3 การจัดการค่าที่หายไปและค่าผิดปกติ

ฟังก์ชัน `clean()` ใน `src/noshow/data/clean.py` ใช้กฎที่ตัดสินใจจากผล EDA (บันทึกเหตุผลไว้ใน `docs/data_decision.txt`)

ปัญหา	จำนวน	การจัดการ	เหตุผล
Age < 0	1	ลบแถว	อายุติดลบเป็นไปไม่ได้ น่าจะเป็นความผิดพลาดในการบันทึก
Age > 100	7	เก็บไว้	อายุสูงสุด 115 ปียังมีความเป็นไปได้
Handicap 2–4	199	แปลงเป็น 1 (1คือมี/0คือไม่มีความพิการ)	ค่า 2–4 มีเพียง 199 จาก 110,527 แถว หรือ 0.18% น้อยเกินกว่าที่โมเดลจะเรียนรู้ความต่างได้
วันจองหลังวันนัด	5	ลบแถว	ต้องจองก่อนถึงวันนัด น่าจะเป็นความผิดพลาดในการบันทึก
เนื้อหาซ้ำ ต่างแค่ AppointmentID	618	เก็บไว้	อาจเป็นการนัดหลายบริการ คิดว่าไม่ผิด
ค่าว่าง	0	schema ปฏิเสธ + pipeline เติมค่า	Pandera ไม่ยอมรับค่าว่างโดยค่าเริ่มต้น ส่วน feature pipeline มี SimpleImputer (median / most_frequent) รองรับกรณีค่าว่างตอนทำนาย

ผลหลัง clean เหลือ **110,521** แถว

อ้างอิง: <src/noshow/data/clean.py>

3.4 การแบ่งชุดข้อมูลตามเวลา

แบ่งตาม `AppointmentDay` แทนการสุ่ม เพราะเมื่อใช้งานจริงระบบต้องทำนายนัดในอนาคตจากข้อมูลในอดีต ถ้าสุ่มแบ่ง โมเดลจะได้เห็นข้อมูลเดือนถัดไประหว่างเทรน (temporal leakage) ทำให้ผลประเมินดีเกินจริง วันที่ใช้แบ่งข้อมูล เก็บใน `configs/params.yaml` และยืนยันจากผล EDA

ชุด	ช่วงวันนัด	จำนวน	สัดส่วน
train	≤ 20 พ.ค. 2016	67,360	60.9%
validation	21–31 พ.ค. 2016	16,711	15.1%
test	≥ 1 มิ.ย. 2016	26,450	23.9%

ชุด test ครอบคลุมเดือนมิถุนายนทั้งเดือน ซึ่งเป็นช่วงที่อัตรา no-show ลดลง จึงใช้ทดสอบได้ว่าโมเดลรับมือข้อมูลที่เปลี่ยนไปได้ ข้อจำกัดคือชุด validation มีวันที่มีข้อมูลจริงเพียง 4 วัน (24, 25, 30, 31 พ.ค.)

ผู้ป่วยซ้ำข้ามชุดข้อมูล: ฟังก์ชัน `patient_overlap()` พบ PatientId ซ้ำระหว่าง train–val 6,564 คน, train–test 9,349 คน และ val–test 3,657 คน ถือว่าปกติเพราะผู้ป่วยกลับมานัดซ้ำได้จริง แต่ feature ประวัติผู้ป่วยต้องนับเฉพาะนัดก่อนหน้านี้เท่านั้นเพื่อป้องกัน leakage (ทีม Modeling นับเฉพาะนัดก่อนวันจอง ดูหัวข้อ 4.1)

```
11
12 > split: # time-based split on AppointmentDay (no random split: avoids temporal leakage)
13   train_end: "2016-05-20" # confirmed by EDA (docs/data_decision.md)
14   val_end: "2016-05-31"
15   group_col: PatientId # check patient overlap across splits
16
```

ภาพที่ 3 วันตัดแบ่งชุดข้อมูลใน `configs/params.yaml`

3.5 การทำซ้ำได้

การแบ่งข้อมูลได้ผลเหมือนเดิมทุกครั้ง เพราะ 1) ข้อมูลเหมือนเดิมตรวจด้วย SHA-256 2) การ clean และการแบ่งข้อมูลมีกฎตายตัว ไม่มีการสุ่ม 3) วันที่ใช้แบ่งข้อมูลอยู่ในไฟล์ config ที่เดียว 4) โลบารรีล็อกเวอร์ชันด้วย `uv.lock` เมื่อรัน 2 ครั้งได้ผลเหมือนกันทุกตัวเลข และผลรวมทั้งสามชุดเท่ากับจำนวนแถวหลัง clean (110,521) จึงไม่มีแถวไหนตกหล่นหรือซ้ำ

```
Problems Output Debug Console Terminal Ports
(clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction> uv run python -c "import pandas as pd, yaml; from noshow.data.clean import clean; from noshow.data.split import time_split, patient_overlap; s=yaml.safe_load(open('configs/params.yaml'))['split']; tr,va,te=time_split(clean(pd.read_csv('data/raw/noshow.csv')), s['train_end'], s['val_end'])); print(len(tr), len(va), len(te), len(tr)+len(va)+len(te)); print(patient_overlap(tr,va,te))"
67360 16711 26450 110521
{'train_val': 6564, 'train_test': 9349, 'val_test': 3657}
(clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction> uv run python -c "import pandas as pd, yaml; from noshow.data.clean import clean; from noshow.data.split import time_split, patient_overlap; s=yaml.safe_load(open('configs/params.yaml'))['split']; tr,va,te=time_split(clean(pd.read_csv('data/raw/noshow.csv')), s['train_end'], s['val_end'])); print(len(tr), len(va), len(te), len(tr)+len(va)+len(te)); print(patient_overlap(tr,va,te))"
67360 16711 26450 110521
{'train_val': 6564, 'train_test': 9349, 'val_test': 3657}
(clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction>
```

ภาพที่ 4 รันการแบ่งข้อมูล 2 ครั้ง ได้ผลเหมือนกัน (67,360 / 16,711 / 26,450 รวม 110,521)

อ้างอิง: <src/noshow/data/split.py>

3.6 Schema และการตรวจจับข้อมูลผิดปกติ

`RAW_SCHEMA` ใน `src/noshow/data/schema.py` ใช้ Pandera กำหนดกฎของข้อมูลดิบทั้ง 14 คอลัมน์ ตรวจ 4 ด้าน: โครงสร้าง (คอลัมน์ครบและไม่เกิน `strict=True`), ชนิดข้อมูล, ช่วงค่า และความสัมพันธ์ (AppointmentID ห้ามซ้ำ)

คอลัมน์	กฎ
PatientId	float, > 0 (เป็น float เพราะมีค่าทศนิยม 5 ค่าในข้อมูลจริง)
AppointmentID	int, ห้ามซ้ำ
Gender	F หรือ M
Age	int, -1 ถึง 120 (ข้อมูลดิบมี -1 อยู่ 1 แถว ซึ่ง clean จะลบออก)
Scholarship, Hipertension, Diabetes, Alcoholism, SMS_received	0 หรือ 1
Handcap	0 ถึง 4
No-show	Yes หรือ No
ScheduledDay, AppointmentDay, Neighbourhood	string

สถิติข้อมูลเสีย: ไฟล์ `tests/fixtures/bad_data.csv` ใส่ Gender = X, Age = 500 และ No-show = Maybe เมื่อป้อนข้อมูลถูกต้อง (`good_data.csv`) ระบบผ่านและแสดง OK เมื่อป้อนข้อมูลเสีย ระบบหยุดทำงานทันที (ไม่แสดง OK) และแจ้งเตือนด้วย `SchemaError` ที่ระบุคอลัมน์ กฎ และค่าที่ผิด

```

● (clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web_app\clinic-noshow-prediction> uv run python -c "import pandas as pd; from noshow.data.schema import RAW_SCHEMA; RAW_SCHEMA.validate(pd.read_csv('tests/fixtures/good_data.csv')); print('OK')"
```

```

OK
○ (clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web_app\clinic-noshow-prediction>
```

ภาพที่ 5 ข้อมูลถูกต้อง *good data.csv* ผ่าน *schema*

```
(clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction> uv run python -c "import pandas as pd; from noshow.data.schema import RAW_SCHEMA; RAW_SCHEMA.validate(pd.read_csv('tests/fixtures/bad_data.csv')); print('OK')"
```

```
ler.py", line 117, in collect_error
    raise schema_error from original_exc
File "C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction\.venv\Lib\site-packages\pandera\backends\pandas\components.py", line 81, in validate_column
    validated_check_obj = super(ColumnBackend, self).validate(
    ~~~~~
File "C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction\.venv\Lib\site-packages\pandera\backends\pandas\array.py", line 139, in validate
    error_handler = self.run_checks_and_handle_errors(
    ~~~~~
File "C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction\.venv\Lib\site-packages\pandera\backends\pandas\array.py", line 203, in run_checks_and_handle_errors
    error_handler.collect_error(
    ~~~~~
File "C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction\.venv\Lib\site-packages\pandera\api\base\error_handler.py", line 117, in collect_error
    raise schema_error from original_exc
pandera.errors.SchemaError: Column 'Gender' failed element-wise validator number 0: isin(['F', 'M']) failure cases
: X
(clinic-noshow-prediction) PS C:\Users\ASUS\Desktop\web app\clinic-noshow-prediction>
```

ภาพที่ 6 ข้อมูลเสีย *bad data.csv* ระบบหยุดและแจ้ง *SchemaError: Gender 'ไม่อยู่ใน' ['F','M'] ค่าที่ผิดคือ X*

การตรวจนี้ถูกใช้ต่อไปใน 3 ที่: unit test (`tests/test_data.py`), CI job `data-validation` (หัวข้อ 7.2) และ
ขั้น validate ใน Prefect pipeline ซึ่งหยุด flow และแจ้งเตือนผ่าน Discord เมื่อข้อมูลไม่ผ่าน (หัวข้อ 8.1)

อ้างอิง: [src/noshow/data/schema.py](#)

3.7 การแปลงข้อมูลชุดเดียวและการป้องกัน Training-Serving Skew

Training-Serving Skew คือการที่ข้อมูลตอนให้บริการถูกแปลงต่างจากตอนเทรน ทำให้โมเดลเจอค่าที่ไม่เคยเห็นและทำนายผิด ทิมป้องกันโดยรวมการแปลงข้อมูลทั้งหมด (สร้าง feature, เติมค่าว่าง, one-hot, scaling) ไว้ใน sklearn Pipeline เดียวจาก `build_preprocessor()` ตอนเทรนรวม pipeline นี้กับโมเดลเป็นออบเจกต์เดียวแล้วบันทึกลง MLflow ตอนให้บริการ API โหลดออบเจกต์เดียวกันจาก Registry แล้วเรียก `predict_proba` กับข้อมูลดิบ ดังนั้นทั้งโค้ดและค่าที่เรียนรู้ระหว่างเทรน (เช่น ค่าเฉลี่ยของ scaler และหมวดหมู่ของ encoder) จึงเหมือนกันทุกประการ

```

13 def build_preprocessor() -> Pipeline:
14     categorical = Pipeline(
15         [
16             ("impute", SimpleImputer(strategy="most_frequent")),
17             ("onehot", OneHotEncoder(handle_unknown="ignore")),
18         ]
19     )
20     numeric = Pipeline(
21         [
22             ("impute", SimpleImputer(strategy="median")),
23             ("scale", StandardScaler()),
24         ]
25     )
26     columns = ColumnTransformer(
27         [("categorical", categorical, CATEGORICAL_FEATURES), ("numeric", numeric, NUMERIC_FEATURES)]
28     )
29     return Pipeline([("features", AppointmentFeatureBuilder()), ("columns", columns)])
30

```

ภาพที่ 7 `build_preprocessor()` โค้ดแปลงข้อมูลชุดเดียว (`src/noshow/features/pipeline.py`)

```

for name, estimator, tune_threshold in experiments(config["seed"]):
    model = Pipeline([("preprocess", build_preprocessor()), ("model", estimator)])
    with mlflow.start_run(run_name=name) as active_run:
        model.fit(X_train, y_train)
        val_probability = model.predict_proba(X_val)[ :, 1]
        if name == selected_name:
            threshold = selected_threshold
        elif tune_threshold:
            threshold = choose_threshold(
                y_val, val_probability, config["model"]["threshold_min_recall"]
            )

```

ภาพที่ 8 ตอนเทรน รวม preprocess กับโมเดลเป็น Pipeline เดียว (`src/noshow/models/train.py`)

```

mlflow.log_params({key: value for key, value in params.items() if value is not
mlflow.log_metrics(metrics)
input_example = X_train.head(3).copy()
input_example["ScheduledDay"] = input_example["ScheduledDay"].astype(str)
input_example["AppointmentDay"] = input_example["AppointmentDay"].astype(str)
model_info = mlflow.sklearn.log_model(
    model,
    name="model",
    input_example=input_example,
    serialization_format=mlflow.sklearn.SERIALIZATION_FORMAT_CLOUDPICKLE,

```

ภาพที่ 9 บันทึก Pipeline ทั้งก้อนด้วย `mlflow.sklearn.log_model`

```
model = mlflow.sklearn.load_model(uri)
```

ภาพที่ 10 ตอนให้บริการ โหลด Pipeline เดียวกันจาก MLflow (`src/noshow/serving/model.py`)

```

else:
    frame = pd.DataFrame.from_records(records)
    scores = self.model.predict_proba(frame)[ :, 1]
    return [

```

ภาพที่ 11 ส่งข้อมูลดิบเข้า Pipeline เพื่อทำนาย

กลไกเสริม: (1) API รับชื่อคอลัมน์เดียวกับข้อมูลดิบ ไม่ต้องแปลงชื่อ (2) ตรวจสอบ input ด้วย Pydantic ที่ใช้ช่วงค่าเดียวกับ Pandera schema และกฎวันจะต้องไม่หลังวันนัดเหมือน `clean()` (3) แปลงเวลาเป็น UTC เหมือนข้อมูลเทรน (4) feature ประดิษฐ์ป้ายนับเฉพาะนัดก่อนหน้า (5) ล็อกเวอร์ชันไลบรารีด้วย `uv.lock` และ Docker image ของ API คัดลอกโค้ด `src/` ชุดเดียวกับที่ใช้เทรน

ข้อจำกัด: กฎ Handcap 2-4 → 1 ยังเขียนซ้ำ 2 ที่ คือ `clean.py` (ตอนเทรน) และ `serving/schemas.py` (ตอนให้บริการ) ผลตอนนี้เหมือนกัน แต่ถ้าแก้ทีเดียวอาจเกิด skew ข้อเสนอแนะคือย้ายกฎนี้เข้าไปใน feature pipeline

4. การพัฒนาโมเดลและการอธิบายผล (ส่วนที่ 3)

งานส่วนนี้: Features & Modeling (นายธันว์ สว่างศรี)

4.1 Features

Feature ทั้งหมดที่ model ใช้		
Feature	ข้อมูลที่ใช้	เหตุผลที่เลือกใช้
Age	อายุเป็นปี	ให้โมเดลเรียนรู้ความสัมพันธ์ของอายุกับ no-show
Gender	เพศ	ให้โมเดลเรียนรู้ความแตกต่างระหว่างกลุ่มเพศ
Neighbourhood	พื้นที่ตามชุดข้อมูล	เพิ่มบริบทพื้นที่ ซึ่งอาจเกี่ยวข้องกับการเข้าถึงบริการ
Scholarship	สถานะ Scholarship 0/1	ให้โมเดลเรียนรู้ความแตกต่างตามสถานะนี้

Feature ทั้งหมดที่ model ใช้		
Feature	ข้อมูลที่ใช้	เหตุผลที่เลือกใช้
Hipertension	ความดันโลหิตสูง 0/1	เพิ่มบริบทสุขภาพที่อาจสัมพันธ์กับการมาตามนัด
Diabetes	เบาหวาน 0/1	เรียนรู้รูปแบบการมาตามนัดของกลุ่มเบาหวาน
Alcoholism	สถานะแอลกอฮอล์ 0/1	ให้โมเดลเรียนรู้ความสัมพันธ์กับ no-show จากข้อมูล
Handcap	สถานะความพิการ 0/1	เพิ่มบริบทข้อจำกัดที่อาจเกี่ยวข้องกับการมารับบริการ
SMS_received	ได้รับ SMS หรือไม่ 0/1	เพิ่มบริบทการเตือนที่มีอยู่ในข้อมูลย้อนหลัง

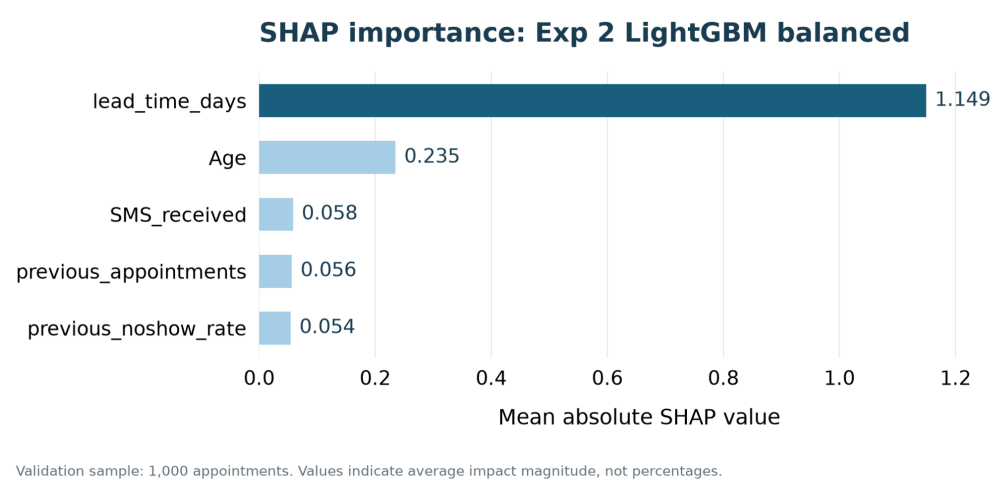
Features ที่เราสร้างเพิ่ม		
5 ตัวที่คำนวณจากวันนัด อายุ และประวัติที่ผ่านมา		
Feature	ข้อมูลที่ใช้	เหตุผลที่เลือกใช้
lead_time_days	วันนัด - วันจอง	สรุประยะเวลาจองนัด เพื่อให้โมเดลเรียนรู้รูปแบบ no-show
appointment_weekday	วันในสัปดาห์ของวันนัด	ให้โมเดลเรียนรู้รูปแบบที่ต่างกันระหว่างวัน
age_group	กลุ่มอายุจาก Age	ให้โมเดลเห็นช่วงวัยร่วมกับอายุจริง
previous_appointments	จำนวนนัดเก่าที่รู้ผลแล้ว	แยกคนไม่มีประวัติกับคนที่มีประวัตินัดหลายครั้ง
previous_noshow_rate	no-show เก่า ÷ นัดเก่า	สรุปพฤติกรรมไม่มาตามนัดในอดีตของแต่ละคน

4.2 การทดลองและการเลือกโมเดล

การทดลองและการเลือกโมเดล						
การทดลอง	Val PR-AUC	Val Recall	Val Precision	Test PR-AUC	Test Recall	Test Precision
Exp 1: Logistic Regression (baseline)	0.2989	71.41%	28.80%	0.2936	63.74%	27.63%
Exp 2: LightGBM balanced (200 trees)	0.3399	80.25%	29.85%	0.3215	75.48%	28.41%
Exp 3: LightGBM tuned (400 trees)	0.3452	60.25%	33.07%	0.3294	54.04%	32.79%

เราเลือก Exp 2: LightGBM balanced เนื่องจากเป้าหมายหลักคือจับผู้ป่วยที่เสี่ยงไม่มาตามนัดให้ได้มาก เพื่อให้คลินิกส่ง SMS หรือโทรเตือนได้ล่วงหน้า โดย Exp 2 มี Recall บนชุด validation สูงที่สุดในสามการทดลองที่ 80.25% และมี PR-AUC เท่ากับ 0.3399 ซึ่งสูงกว่า Logistic Regression ที่ใช้เป็น baseline แม้ Exp 3 จะมี PR-AUC และ Precision สูงกว่าเล็กน้อย แต่มี Recall ต่ำกว่าภายใต้การตั้งค่าและ threshold ของการทดลองนั้น จึงเลือก Exp 2 ให้สอดคล้องกับเป้าหมายที่เน้นลดการพลาดผู้ป่วยกลุ่มเสี่ยง และรักษาสล็อตแพทย์เพื่อให้คลินิกสามารถรักษาต้นทุนหรือรายได้หลักไว้ได้

4.3 การอธิบายผล (SHAP)



เราใช้ SHAP อธิบายการทำนายของ Exp 2 โดยสุ่มนัดจากชุด validation มา 1,000 นัด พบว่าระยะเวลาตั้งแต่จองจนถึงวันนัดมีผลต่อคะแนนทำนายมากที่สุดในตัวอย่างนี้ รองลงมาคืออายุ ส่วนการได้รับ SMS จำนวนนัดที่ผ่านมา และอัตราไม่มาตามนัดในอดีต

กราฟนี้บอกว่าโมเดลใช้ข้อมูลอะไรในการทำนายมากที่สุด แต่ยังบอกไม่ได้ว่าค่าสูงทำให้เสี่ยงเพิ่มขึ้นหรือลดลง และไม่ได้ยืนยันว่าปัจจัยนั้นเป็นสาเหตุของการไม่มาตามนัด

5. การติดตามการทดลองและทะเบียนโมเดล (ส่วนที่ 4)

งานส่วนนี้: Tracking, Registry & Pipeline (นายกิตตินันท์ ไชไพรัตน์)

5.1 บันทึกการทดลองครบ 6 อย่าง

สิ่งที่ต้องบันทึก	หลักฐานใน MLflow
เวอร์ชันโค้ด	Git SHA เป็น tag <code>git_sha</code>
เวอร์ชันข้อมูล	SHA-256 ของ <code>data/raw/noshow.csv</code> เป็น tag <code>data_sha256</code>
ไฮเปอร์พารามิเตอร์	ค่าจาก <code>estimator.get_params()</code>
ตัวชี้วัด	validation/test PR-AUC, no-show recall, precision, threshold
ไฟล์ผลลัพธ์	serialized pipeline, <code>metrics.json</code> , <code>data_split.json</code> , SHAP ของ Exp 2
สภาพแวดล้อม	<code>pyproject.toml</code> , <code>uv.lock</code> และ environment ของ model artifact

บันทึกการทดลอง 3 แบบ ได้แก่ Logistic Regression, LightGBM balanced และ LightGBM tuned เพื่อเปรียบเทียบผลและตรวจสอบย้อนหลังได้ว่าแต่ละโมเดลสร้างจากโค้ด ข้อมูล และการตั้งค่าใด

Experiments > clinic-noshow-modeling >

☰

Runs ⓘ

☰

📈

📊

🔍

metrics.rmse < 1 and params.model = "tree"

⌚

⌚

All time ▾

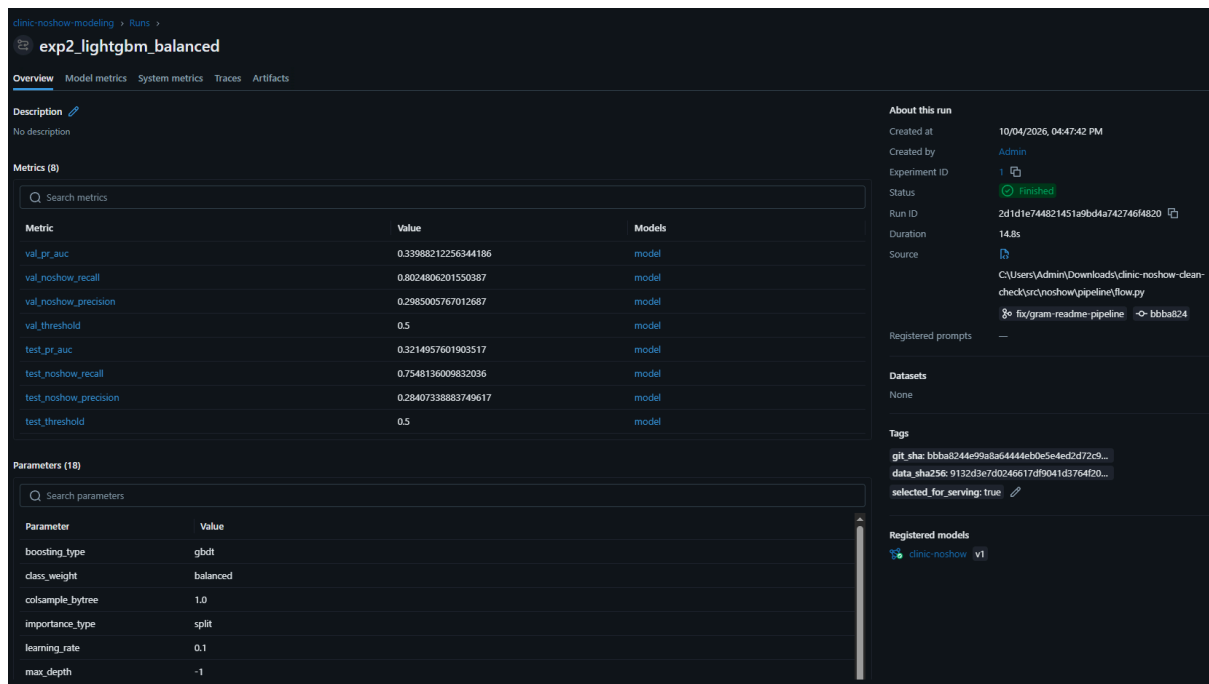
State: Active ▾

Datasets

			Metrics				
☐	Run Name	Created ⌵	Duration	val_noshow_pre	val_noshow_rec	val_pr_auc	val_threshold
☐	exp3_lightgbm_tuned	✔ 5 hours ago	2.1min	0.33066712...	0.60248062...	0.34516874...	0.59784381...
☐	exp2_lightgbm_balanced	✔ 5 hours ago	14.8s	0.29850057...	0.80248062...	0.33988212...	0.5
☐	exp1_logistic_regression	✔ 5 hours ago	8.9s	0.28801900...	0.71410852...	0.29890403...	0.5

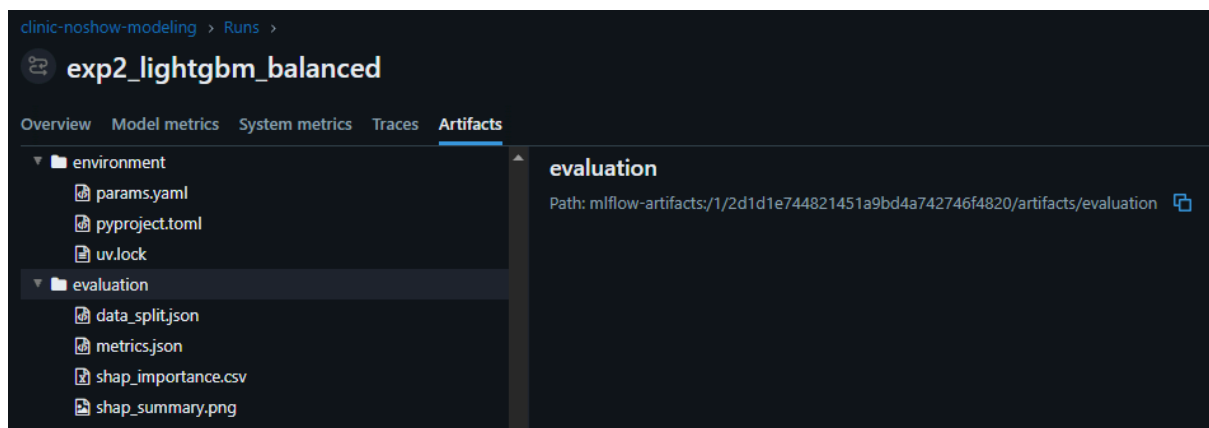
ภาพที่ 1 การเปรียบเทียบผลการทดลองทั้ง 3 โมเดลใน MLflow

ระบบใช้ MLflow บันทึกและเปรียบเทียบการทดลอง ได้แก่ Logistic Regression, LightGBM แบบ balanced และ LightGBM แบบ tuned โดยพิจารณา PR-AUC, no-show recall, precision และ threshold บนชุด validation เดียวกัน ทีมเลือก LightGBM แบบ balanced เนื่องจากมี recall ประมาณ 0.8025 สูงกว่าอีกสองโมเดล และมี PR-AUC ประมาณ 0.3399 ผ่านเกณฑ์ขั้นต่ำ สอดคล้องกับเป้าหมายการค้นหาผู้เสี่ยงไม่มาตามนัด เพื่อส่งข้อความหรือโทรเตือน อย่างไรก็ตาม precision อยู่ที่ประมาณ 0.2985 จึงต้องพิจารณาภาระและค่าใช้จ่ายในการแจ้งเตือนร่วมด้วย



ภาพที่ 2 รายละเอียดการทดลองและข้อมูลสำหรับตรวจสอบย้อนกลับ

หน้า Overview ของการทดลอง exp2_lightgbm_balanced แสดงตัวชี้วัดบนชุด validation และ test พร้อมไฮเปอร์พารามิเตอร์ของโมเดล การทดลองจบในสถานะ Finished และบันทึก Run ID สำหรับอ้างอิง นอกจากนี้ยังมี tags ได้แก่ git_sha ซึ่งระบุเวอร์ชันโค้ด และ data_sha256 ซึ่งระบุข้อมูลด้วยค่า SHA-256 ทำให้สามารถตรวจสอบได้ว่าผลการทดลองเกิดจากโค้ดและข้อมูลชุดใด รวมถึงเชื่อมโยงไปยังโมเดลที่ลงทะเบียนใน Registry ได้



ภาพที่ 3 ไฟล์ผลลัพธ์และสภาพแวดล้อมที่บันทึกเป็น Artifacts

ระบบบันทึกไฟล์ประกอบการทดลองไว้ใน MLflow เพื่อให้ตรวจสอบผลและเตรียมสภาพแวดล้อมสำหรับรันซ้ำได้ โพลเดอร์ environment ประกอบด้วย params.yaml, pyproject.toml และ uv.lock สำหรับเก็บการตั้งค่าและ dependencies ส่วนโพลเดอร์ evaluation ประกอบด้วย data_split.json, metrics.json, shap_importance.csv และ shap_summary.png สำหรับแสดงการแบ่งข้อมูล ผลการประเมิน และการอธิบายความสำคัญของ features เมื่อรวมกับข้อมูลในหน้า Overview จึงครอบคลุมการบันทึกเวอร์ชันโค้ด เวอร์ชันข้อมูล ไฮเปอร์พารามิเตอร์ ตัวชี้วัด ไฟล์ผลลัพธ์ และสภาพแวดล้อมที่ใช้รัน

5.2 Model Registry, ด้านตรวจ และการย้อนกลับ

- โมเดลชื่อ `clinic-noshow` ใน MLflow Registry ใช้ alias `challenger`, `champion` และ `previous_champion`
- Gate (`noshow.registry.gate`) อ่านเกณฑ์จาก `configs/slo.yaml` ประเมิน challenger และ champion บน validation ชุดเดียวกัน ถ้า `must_beat_production: true` challenger ต้องมี recall สูงกว่า champion จึงผ่าน
- Rollback (`noshow.registry.rollback`) ย้าย alias `champion` กลับไปยังเวอร์ชันที่ `previous_champion` ซ้ำอยู่ และหากระบุ `--api-url` จะเรียก `/reload` ของ API พร้อมตรวจว่าเวอร์ชันที่โหลดตรงกับเวอร์ชันหลัง rollback

Model Registry และ Gate

ลงทะเบียนโมเดลใน MLflow Registry ภายใต้ชื่อ `clinic-noshow` โดยใช้ alias ตามข้อตกลงทีมดังนี้

Alias	ความหมาย
challenger	โมเดลที่รอการตรวจ Gate
champion	โมเดลที่อนุมัติให้ระบบใช้
previous_champion	เวอร์ชัน champion ก่อนหน้าสำหรับ rollback

Gate ใน `noshow.registry.gate` อ่านเกณฑ์ได้จาก `configs/slo.yaml` และตรวจความพร้อมของ model version สถานะการฝึก ความถูกต้องของ threshold รวมถึงความสอดคล้องของข้อมูลและการแบ่งชุดข้อมูล

เกณฑ์	เงื่อนไข
Validation PR-AUC	ไม่น้อยกว่า 0.298904
Validation no-show recall	ไม่น้อยกว่า 0.60
ขนาดโมเดล	ไม่เกิน 100 MB
เมื่อมี champion และ <code>must_beat_production</code> เป็น true	ประเมินทั้งสองโมเดลบน validation ชุดเดียวกัน และ challenger ต้องมี recall สูงกว่า champion

สำหรับโมเดลแรกที่ยังไม่มี champion ระบบตรวจเกณฑ์ขั้นต่ำโดยไม่เปรียบเทียบกับโมเดล production เมื่อ Gate ผ่าน promote จะเก็บ champion เดิมไว้เป็น `previous_champion` หากมีเวอร์ชันเดิมที่ต่างกัน แล้วเปลี่ยน champion ให้ชี้ไปยังโมเดลใหม่

ผลทดสอบในเครื่อง: challenger ที่ recall เท่ากับ champion ไม่ถูก promote, demo ผ่าน gate แล้ว API เปลี่ยน version 1 → 2 และ rollback แล้ว API กลับเป็น version 2 → 1

Registered Models >

clinic-noshow

Created Time: 10/04/2026, 04:49:49 PMLast Modified: 10/04/2026, 04:49:50 PM

> Description Edit

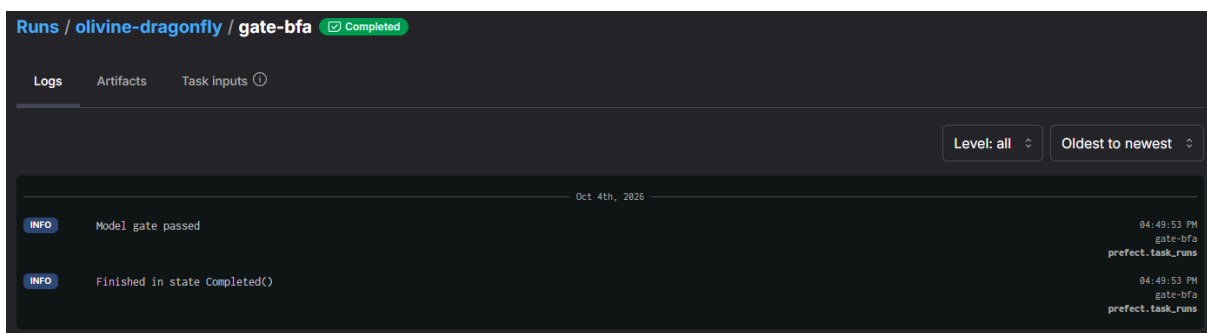
> Tags

> Versions Compare

Version	Registered at	Created by	Tags	Aliases
Version 1	10/04/2026, 04:49:50 PM		threshold: 0.5 selected_experiment: exp2_lightgbm_balanced original_mo...: models/m-92332b63572845a092c... git_sha: bbb8244e99a8a64444eb0e5e4ed2d72c9... data_sha256: 9132d3e7d0246617df9041d3764f20...	@ challenger @ champion

ภาพที่ 4 การจัดการเวอร์ชันโมเดลใน MLflow Model Registry

ระบบใช้ MLflow Model Registry จัดเก็บโมเดลชื่อ clinic-noshow และกำหนด alias challenger สำหรับโมเดลที่รอการตรวจ และ champion สำหรับโมเดลที่อนุมัติให้ใช้งาน การทดสอบครั้งแรกลงทะเบียน LightGBM แบบ balanced เป็น Version 1 พร้อม tags เช่น threshold, selected_experiment, git_sha และ data_sha256 หลังผ่าน Gate ระบบกำหนด champion ให้ชื่อ Version 1 จึงเห็นทั้ง challenger และ champion ซีเวอร์ชันเดียวกัน API อ้างอิงโมเดลผ่าน models:/clinic-noshow@champion เพื่อโหลดเวอร์ชันที่ได้รับอนุมัติ



ภาพที่ 5 การตรวจ Gate ก่อนอนุมัติโมเดลขึ้นใช้งาน

ระบบตรวจความพร้อมของโมเดลและการทดลอง ความสอดคล้องของข้อมูลและ threshold รวมถึงเกณฑ์ PR-AUC ไม่น้อยกว่า 0.298904, no-show recall ไม่น้อยกว่า 0.60 และขนาดโมเดลไม่เกิน 100 MB หากมี champion อยู่แล้ว จะประเมินทั้งสองโมเดลบนชุด validation เดียวกันและกำหนดให้ challenger มี recall สูงกว่า champion จึงอนุมัติ การทดสอบรอบแรกยังไม่มี champion สำหรับเปรียบเทียบ และโมเดลผ่านเกณฑ์ที่กำหนด จึงแสดงข้อความ Model gate passed ก่อนดำเนินการ promote และ deploy

5.3 Rollback และการตรวจ API reload

Rollback (noshow.registry.rollback) ตรวจสอบว่า previous_champion มีอยู่ มีสถานะ READY และมี threshold ที่ถูกต้อง ก่อนย้าย alias champion กลับไปยังเวอร์ชันที่ previous_champion ซ้ำอยู่ หากเรียกผ่าน CLI พร้อมระบุ --api-url จะเรียก /reload ของ API และตรวจสอบว่า model_version ที่ API ตอบกลับตรงกับเวอร์ชันหลัง rollback

ฟังก์ชัน rollback เปลี่ยนเฉพาะ alias champion ไม่ได้สลับ champion กับ previous_champion ดังนั้นในการทดสอบ v2 → v1 หลัง rollback ทั้ง champion และ previous_champion จะชี้ v1

```

(clicin-noshow-prediction) PS C:\Users\Admin\Downloads\clinic-noshow-clean-check> uv run python scripts/check_rollback.py
warning: `VIRTUAL_ENV=C:\Users\Admin\Downloads\clinic-noshow-work\.venv` does not match the project environment path `\.venv` and will be ignored; use `--active`
to target the active environment instead
Successfully registered model 'clinic-noshow-rollback-demo-4f6fed6'.
Copied version '1' of model 'clinic-noshow' to version '1' of model 'clinic-noshow-rollback-demo-4f6fed6'.
Registered model 'clinic-noshow-rollback-demo-4f6fed6' already exists. Creating a new version of this model...
Copied version '1' of model 'clinic-noshow' to version '2' of model 'clinic-noshow-rollback-demo-4f6fed6'.
Before rollback: clinic-noshow-rollback-demo-4f6fed6, champion=2
{
  "model_name": "clinic-noshow-rollback-demo-4f6fed6",
  "rolled_back_from": "2",
  "champion_version": "1",
  "model_uri": "models:/clinic-noshow-rollback-demo-4f6fed6@champion",
  "threshold": 0.5
}
ROLLBACK VERIFIED
(clicin-noshow-prediction) PS C:\Users\Admin\Downloads\clinic-noshow-clean-check>

```

ภาพที่ 6 การสาธิตย้อนกลับไปยังโมเดลเวอร์ชันก่อนหน้า

การทดสอบสร้างโมเดลแยกชื่อ clinic-noshow-rollback-demo-4f6fed6 และคัดลอกโมเดลหลักเป็น demo Version 1 และ Version 2 โดยกำหนด champion ให้ชื่อ Version 2 และ previous_champion ให้ชื่อ Version 1 เมื่อเรียก rollback ระบบเปลี่ยน champion กลับไปยัง Version 1 และตรวจสอบ alias หลังเปลี่ยนสำเร็จ ผลแสดง rolled_back_from เป็น 2, champion_version เป็น 1 และข้อความ ROLLBACK VERIFIED การทดสอบยืนยันการย้อนเวอร์ชันใน Registry โดยทั้งสองเวอร์ชันเป็นสำเนาของโมเดลเดียวกัน ไม่เปลี่ยน champion ของโมเดลหลัก และไม่ได้ทดสอบ API reload ในรอบนี้

```

(clicin-noshow-prediction) PS C:\Users\Admin\Downloads\clinic-noshow-clean-check> $env:PREFECT_API_URL = "http://127.0.0.1:4200/api"
>> uv run python scripts/check_deployment_cycle.py --model-name "clinic-noshow-integration-demo-d2f3549f" --api-url "http://127.0.0.1:8001"
warning: `VIRTUAL_ENV=C:\Users\Admin\Downloads\clinic-noshow-work\.venv` does not match the project environment path `\.venv` and will be ignored; use `--active`
to target the active environment instead
15:32:18.521 | INFO | Flow run 'festive-anaconda' - Beginning flow run 'festive-anaconda' for flow 'deployment-cycle-demo'
15:32:18.531 | INFO | Flow run 'festive-anaconda' - View at http://127.0.0.1:4200/runs/flow-run/8fa71c38-e071-4a84-b463-28b3a1d75d17
Downloading artifacts: 100% | 1/1 [00:00:00:00, 12.49it/s]
Downloading artifacts: 100% | 7/7 [00:00:00:00, 46.63it/s]
Downloading artifacts: 100% | 1/1 [00:00:00:00, 18.33it/s]
Downloading artifacts: 100% | 7/7 [00:00:00:00, 59.54it/s]
15:32:23.007 | INFO | Task run 'deploy-a4d' - API loaded champion version 2
15:32:24.072 | INFO | Task run 'deploy-a4d' - Finished in state Completed()
15:32:24.076 | INFO | Flow run 'festive-anaconda' - PROMOTE + RELOAD VERIFIED {'previous_version': '1', 'model_version': '2'}
15:32:24.940 | INFO | Task run 'deploy-b3e' - API loaded champion version 1
15:32:24.944 | INFO | Task run 'deploy-b3e' - Finished in state Completed()
15:32:24.948 | INFO | Flow run 'festive-anaconda' - ROLLBACK + RELOAD VERIFIED {'previous_version': '2', 'model_version': '1'}
15:32:24.970 | INFO | Flow run 'festive-anaconda' - DEPLOYMENT CYCLE VERIFIED; main champion unchanged
15:32:25.547 | INFO | Flow run 'festive-anaconda' - Finished in state Completed()
(clicin-noshow-prediction) PS C:\Users\Admin\Downloads\clinic-noshow-clean-check>

```

ภาพที่ 7 ทดสอบวงจร promote → reload → rollback → reload บนโมเดล demo แยก

การทดสอบเพิ่มเติมใช้โมเดล clinic-noshow-integration-demo-d2f3549f และ API demo ที่พอร์ต 8001 โดยสคริปต์ check_deployment_cycle.py เรียก promote แล้ว deploy_model เพื่อ reload API จาก v1 เป็น v2 จากนั้นเรียก rollback แล้ว deploy_model อีกครั้งเพื่อโหลดกลับ v1 และตรวจผลผ่าน /health

ผลแสดง PROMOTE + RELOAD VERIFIED, ROLLBACK + RELOAD VERIFIED และ DEPLOYMENT CYCLE VERIFIED; main champion unchanged โดย flow จบในสถานะ Completed ใช้เวลาประมาณ 7 วินาที ผลนี้ยืนยันการเปลี่ยนเวอร์ชันและ reload จริงบนโมเดล demo ไม่ใช่การปรับ champion ของโมเดลหลัก

6. การให้บริการและโครงสร้างพื้นฐาน (ส่วนที่ 5)

งานส่วนนี้: Serving & Infra (นายภูรินทร์ ศรีฐาน)

อ้างอิง [docs/serving.md](#) และ [docs/loadtest_report.md](#)

6.1 API ในคอนเทนเนอร์

Method	Path	หน้าที่
GET	/health	ตรวจสอบสุขภาพระบบ + version/threshold ของโมเดลที่โหลดอยู่
POST	/predict	ทำนาย 1 นัดหมาย
POST	/predict_batch	ทำนาย 1–1,000 นัดหมาย
POST	/reload	โหลด champion ใหม่ทันที
GET	/metrics	Prometheus metrics (จำนวน request, latency)

API รันใน Docker ([docker/Dockerfile.api](#)) ด้วย uvicorn 2 workers มี HEALTHCHECK เรียก `/health` ทุก 15 วินาที และเปิดพร้อมบริการอื่นด้วย `docker compose up` ข้อมูลที่ผิดรูปแบบถูกปฏิเสธด้วย HTTP 422 และถ้ายังไม่มีโมเดลจะตอบ 503

Clinic No-Show Prediction API 0.1.0 CAS 3.1

/openapi.json

default

GET	/health	Health
POST	/predict	Predict
POST	/predict_batch	Predict Batch
POST	/reload	Reload
GET	/metrics	Metrics

ภาพที่ 12 หน้า /docs ของ API แสดง endpoint ทั้ง 5

```
{
  "status": "ok",
  "version": "0.1.0",
  "model_loaded": true,
  "model_version": "1",
  "model_uri": "models:/clinic-noshow@champion",
  "threshold": 0.5,
  "last_error": null,
  "pid": 10,
  "workers": 2,
  "profiling": false
}
```

ภาพที่ 13 ผล GET `/health` แสดงว่าโหลดโมเดล champion เวอร์ชัน 1 แล้ว ใช้ 2 workers

ภาพที่ 14 เรียก POST /predict ด้วยข้อมูลนัด 1 รายการ ได้คะแนนความเสี่ยง 0.56 และ alert = true (threshold 0.5)

6.2 รูปแบบการให้บริการ

เลือก **real-time (online) inference** ผ่าน **/predict** เพราะเจ้าหน้าที่ต้องรู้ความเสียนทันทีที่นัดถูกสร้าง ร่วมกับ **batch** ผ่าน **/predict_batch** สำหรับประมวลผลนัดของวันถัดไปทั้งหมด และใช้ **micro-batching** (สูงสุด 32 แถว) รวม request ที่มาพร้อมกันเป็นการเรียก pipeline ครั้งเดียว เพราะ profiling พบว่า preprocessing ใช้เวลาประมาณ 87% และเป็นต้นทุนคงที่ต่อการเรียก 1 ครั้ง

6.3 Latency, Throughput และ SLO

ทดสอบด้วย Locust 50 users นาน 2 นาที (traffic **/predict** 10 : **/predict_batch** 1 : **/health** 1) เทียบกับ SLO ใน **configs/slo.yaml**

SLO	วัดได้	เป้าหมาย	ผล
Latency p50 /predict	45 ms	≤ 50 ms	PASS
Latency p95 /predict	80 ms	≤ 200 ms	PASS
Error rate	0.00%	$\leq 1.00\%$	PASS
Availability	100.00%	$\geq 99.00\%$	PASS
Throughput (รวมทุก endpoint)	142.5 req/s	—	—

รอบในตารางวัดโดยให้ Locust อยู่ใน container ในเครือข่าย Docker เดียวกับ API ใช้ 2 workers + micro-batching 32 หลัง warm-up และวัดรอบเดียว หากวัดจาก Windows ด้วยการตั้งค่าเดียวกันจะได้ p50 74 ms (ไม่ผ่าน) เพราะ port forwarding ของ Docker Desktop เพิ่มเวลาราว 46 ms ต่อคำขอ p50 จึงผ่านแบบเฉียด เหลือระยะเผื่อเพียง 5 ms

ก่อนและหลังปรับปรุง:

การตั้งค่า	p50	p95	req/s	SLO
1 worker (baseline)	1100 ms	1500 ms	35.7	FAIL
2 workers	160 ms	300 ms	107.0	FAIL
4 workers	75 ms	170 ms	130.0	p95 PASS
2 workers + batching 32 ใน(windows)	74 ms	99ms	133.	p95 PASS
2 workers + batching 32 (client ใน Docker network)	45 ms	80 ms	142.5	PASS

Load test report (Serving)

- Date: 2026-10-03 22:09
- Configuration: 50 users, inside Docker network, after warm-up
- Tool: Locust, 50 concurrent users, duration 2m
- Traffic mix: /predict 10 : /predict_batch (20 rows) 1 : /health 1
- Raw results: [loadtest/results/net50b_stats.csv](#)

SLO check: 50 users, inside Docker network, after warm-up (targets from `configs/slo.yaml`)

SLO	Measured	Target	Result
p50 latency /predict	45 ms	<= 50 ms	PASS
p95 latency /predict	80 ms	<= 200 ms	PASS
error rate (all requests)	0.00%	<= 1.00%	PASS
availability	100.00%	>= 99.00%	PASS

Overall: all SLOs met

Per-endpoint latency: 50 users, inside Docker network, after warm-up

Endpoint	Requests	Req/s	p50 (ms)	p95 (ms)	p99 (ms)	Error rate
GET /health	1421	11.9	11	24	35	0.00%
POST /predict	14224	119.1	45	80	120	0.00%
POST /predict_batch	1369	11.5	47	78	110	0.00%
Aggregated	17014	142.5	44	78	110	0.00%

ภาพที่ 15 ผลเทียบ SLO รอบในเครือข่าย Docker (50 users, 2 workers + micro-batching 32, หลัง warm-up)

อ้างอิง: [docs/loadtest_report.md](#)

7. การเฝ้าระวังและ CI/CD (ส่วนที่ 6)

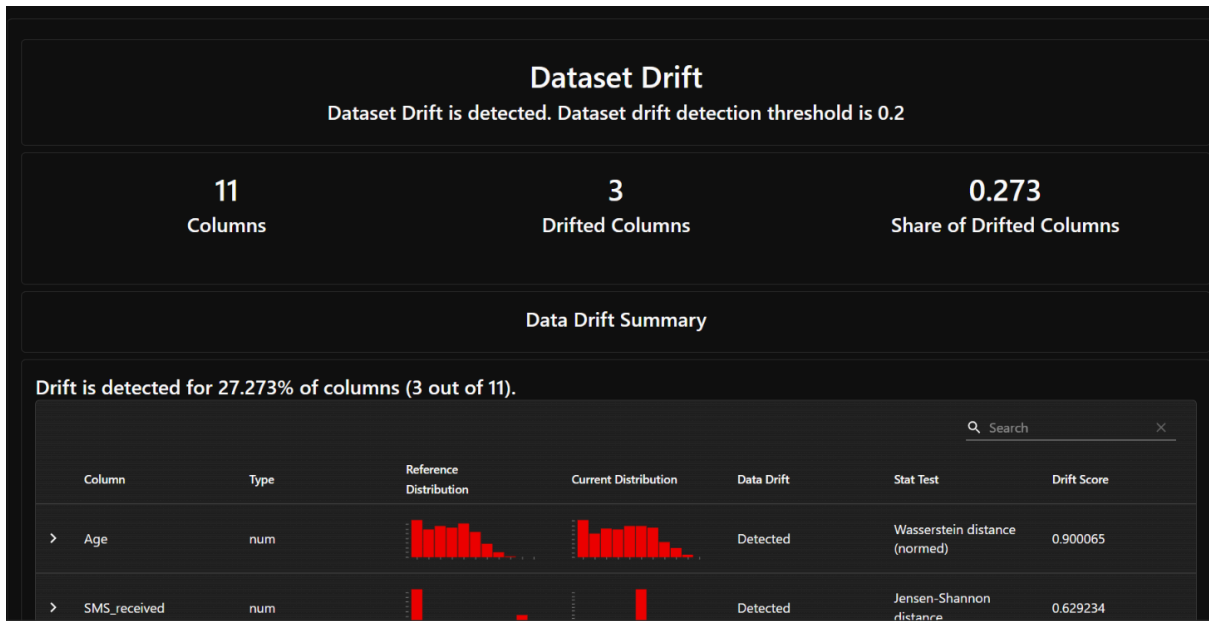
7.1 ระบบเฝ้าระวัง

ด้านสถานะระบบ: API ส่ง metrics ที่ `/metrics` (จำนวน request แยก path/status และ latency histogram) ให้ Prometheus เก็บ และ Grafana แสดงผล

ชุดข้อมูลจำลอง drift (จากงาน Data & Validation) สร้างด้วย `scripts/make_drift.py` จากชุด test:

ไฟล์	Feature	ผลลัพธ์ (No-show)	ใช้ทดสอบ
reference.csv	ปกติ (ชุด train)	ปกติ	จุดเทียบ
data_drift.csv	เปลี่ยน: อายุ +20 ปี, ทุกคนได้ SMS	ปกติ	ระบบจับ data drift ได้หรือไม่
concept_drift.csv	ปกติ	เปลี่ยน: ผู้ที่ได้ SMS ไม่มาทั้งหมด	ระบบจับ concept drift ได้หรือไม่

Data drift ตรวจได้ทันทีจากการกระจายของ feature ส่วน concept drift feature เหมือนเดิม ต้องรอผลจริง แล้วพบว่าคุณภาพการทำนาย (recall/PR-AUC) ลดลง



ผลการตรวจสอบ Data Drift ด้วย Evidently พบ Drift จำนวน 3 จาก 11 features คิดเป็น 27.27% ซึ่งสูงกว่าเกณฑ์ 20% จึงตรวจพบ Dataset Drift โดยตัวอย่าง feature ที่เกิด Drift ได้แก่ Age และ SMS_received



Grafana Monitoring Dashboard แสดงการเฝ้าระวังระบบ ได้แก่ Throughput, API Latency, API Error Rate และ Prediction Distribution โดยดึง Metrics ที่ Prometheus จัดเก็บมาแสดงผล

```
configs > {} monitoring_policy.json > ...
1  {
2    "reference_csv": "data/processed/reference.csv",
3    "current_csv": "data/processed/data_drift.csv",
4    "feedback_csv": "data/processed/feedback.csv",
5    "report_dir": "reports/monitoring",
6    "feature_psi_threshold": 0.2,
7    "drift_share_threshold": 0.2,
8    "min_labeled_rows": 200,
9    "min_roc_auc": 0.7,
10   "max_roc_auc_drop": 0.05,
11   "baseline_roc_auc": null,
12   "sms_effect_delta_threshold": 0.1,
13   "min_sms_group_rows": 30,
14   "prediction_positive_rate_delta_threshold": 0.1,
15   "p95_latency_seconds": 0.3,
16   "error_rate_threshold": 0.01,
17   "score_threshold": 0.5
18 }
19
```

ภาพที่ 3 เกณฑ์การแจ้งเตือนและ Monitoring Policy กำหนด Threshold สำหรับตรวจสอบ Data Drift, Concept/Performance Drift และประสิทธิภาพของระบบ เพื่อใช้ประกอบการตัดสินใจเฝ้าระวังและ Retrain โมเดล

```

if concept_drift:
    action = "RETRAIN"
    reason = "Concept/performance drift confirmed with enough delayed labels. Run the existing train-evaluate-gate pipeline."
elif data_drift_alert or prediction_drift_alert:
    action = "WATCH"
    reason = "Input/prediction drift detected. Investigate and collect delayed labels; do not retrain on unlabeled drift alone."
elif label_state == "not_enough_labels":
    action = "WAIT_FOR_LABELS"
    reason = "Not enough delayed labels for a reliable performance/concept-drift decision."
else:
    action = "OK"
    reason = "No alert crossed the configured thresholds."

```

ภาพที่ 4 นโยบายการตัดสินใจ Retrain โมเดล ระบบจะ Retrain เมื่อยืนยัน Concept/Performance Drift จากข้อมูลที่มี Label เพียงพอ ส่วน Data/Prediction Drift จะเข้าสู่สถานะ WATCH เพื่อเฝ้าระวังและรวบรวม Label เพิ่ม โดยไม่ Retrain จาก Data Drift เพียงอย่างเดียว

```

22:55:09.115 | WARNING | Task run 'gate-d84' - Model will not be promoted: [Recall must be higher than champion recall]
22:55:09.119 | INFO | Task run 'gate-d84' - Finished in state completed()
22:55:10.138 | INFO | Flow run 'convivial-wren' - Finished in state completed()
{'validated_rows': 110527, 'train_rows': 67360, 'val_rows': 16711, 'test_rows': 26450, 'feature_count': 105, 'trained_models': 3, 'selected_run_id': 'df46f7bb9ca8498291589c1cb62e5c96',
'challenger_version': '3', 'gate_passed': False, 'promotion_status': 'not_promoted', 'deployment': None}

```

ภาพที่ 5 ผลการสาธิตวงจร Retrain และ Model Gate หลังจาก Retrain จำนวน 3 โมเดล ระบบสร้าง Challenger Version 3 และนำเข้าสู่ Model Gate พบว่า Recall ของ Challenger ไม่สูงกว่า Champion จึงไม่ผ่าน Gate (`gate_passed = False`) และไม่ถูก Promote หรือ Deploy ทำให้ระบบยังคงใช้ Champion เดิมใน Production

7.2 CI/CD

งานส่วนนี้: CI/CD & Testing (นายณันทพงศ์ พยัคฆะเริง) อ้างอิง [docs/ci.md](#) workflow [.github/workflows/ci.yml](#) รันทุก Pull Request และทุก push เข้า `main` มี 3 jobs ตามแนวคิดทดสอบ Code, Data, Model

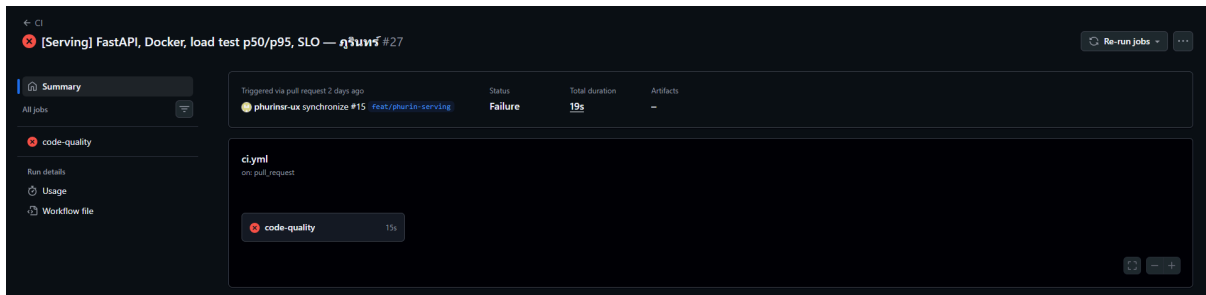
Job	ตรวจอะไร	ไม่ผ่านเมื่อ
code-quality	ruff check, ruff format --check, pytest	โค้ดผิดกฎหรือ test fail
data-validation	good_data ต้องผ่าน, bad_data และ bad_data_cases ต้องถูกปฏิเสธ	schema หลวมเกินจนข้อมูลเสียหลุดผ่าน
model-gate	PR-AUC ≥ 0.298904 และ recall ≥ 0.60 จาก docs/model_results.json	โมเดลต่ำกว่าเกณฑ์

`model-gate` รันหลังสอง job แรกผ่านแล้วเท่านั้น (`needs`) และทุก job ติดตั้งไลบรารีด้วย `uv sync --frozen` จึงได้สภาพแวดล้อมเดิมทุกครั้ง

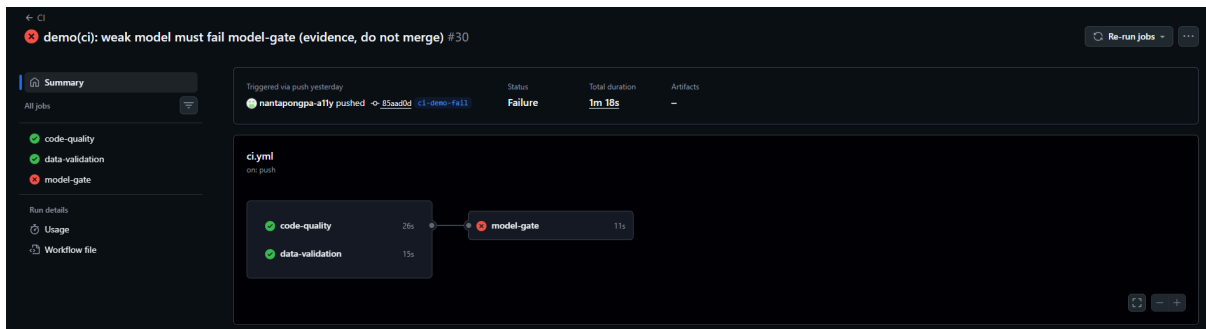
หลักฐานครั้งที่ผ่าน

ภาพที่ 16 CI ผ่านทั้ง 3 jobs หลัง merge PR #17

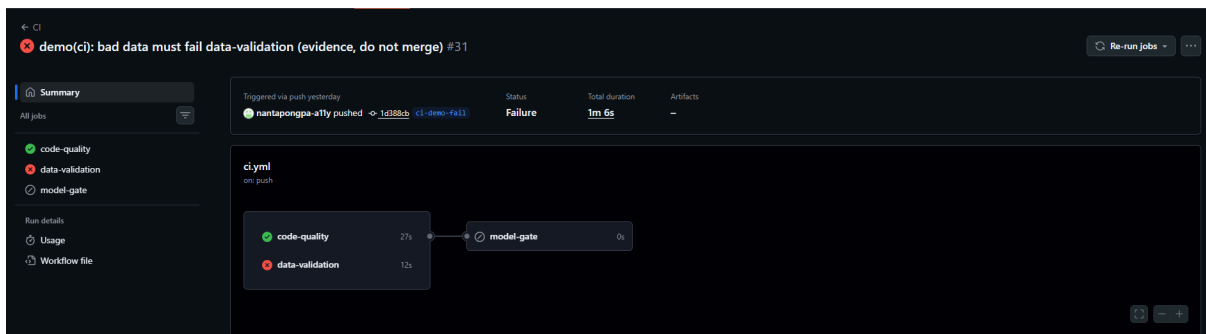
หลักฐานครั้งที่ไม่ผ่าน



ภาพที่ 17 CI จับปัญหาจริงใน PR ของ Serving (code-quality fail)



ภาพที่ 18 ทดสอบป้อนโมเดลที่แย่ model-gate ไม่ผ่าน



ภาพที่ 19 ทดสอบป้อนข้อมูลเสีย data-validation ไม่ผ่าน และ model-gate ถูกข้าม

การทดสอบในเครื่อง

```
(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run ruff check .
All checks passed!

(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run python -m noshow.ci.data_validation test
s/fixtures/good_data.csv
VALID: tests\fixtures\good_data.csv (0 problem(s), expected VALID)

(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run python -m noshow.ci.data_validation test
s/fixtures/bad_data_cases.csv
INVALID: tests\fixtures\bad_data_cases.csv (7 problem(s), expected VALID)
- column=PatientId check=greater_than(0) value=-5.0
- column=AppointmentID check=field_uniqueness value=6000004.0
- column=AppointmentID check=field_uniqueness value=6000004.0
- column=Age check=in_range(-1, 120) value=150.0
- column=Scholarship check=isin([0, 1]) value=2.0
- column=Handcap check=in_range(0, 4) value=7.0
- column=SMS_received check=isin([0, 1]) value=3.0
!! unexpected result for tests\fixtures\bad_data_cases.csv
```

ภาพที่ 16 ตรวจสอบข้อมูลดีและข้อมูลเสียด้วย *noshow.ci.data_validation* (ข้อมูลเสียพบ 7 ปัญหา)

```
(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run python -m noshow.ci.model_gate
Model: exp2_lightgbm_balanced (from model_results.json)
PR-AUC: 0.33988212256344186 (gate >= 0.298904)
Recall: 0.8024806201550387 (gate >= 0.6)
Model gate PASSED

(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run python -m noshow.ci.model_gate --results
tests/fixtures/bad_model_results.json
Model: exp2_lightgbm_balanced (from bad_model_results.json)
PR-AUC: 0.25 (gate >= 0.298904)
Recall: 0.55 (gate >= 0.6)
Model gate FAILED:
- PR-AUC 0.2500 < 0.298904
- no-show recall 0.5500 < 0.6

(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run python -m noshow.ci.data_validation tests
s/fixtures/good_data.csv
INVALID: tests\fixtures\good_data.csv (1 problem(s), expected VALID)
- column=Gender check=isin(['F', 'M']) value='X'
!! unexpected result for tests\fixtures\good_data.csv
```

ภาพที่ 17 model gate: โมเดลที่เลือกผ่าน, โมเดลแย่ไม่ผ่าน และข้อมูลที่ถูกแก้ไขให้ผิดถูกจับได้

```
(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run python -m noshow.ci.model_gate
Model: exp2_lightgbm_balanced (from model_results.json)
PR-AUC: 0.33988212256344186 (gate >= 0.5)
Recall: 0.8024806201550387 (gate >= 0.9)
Model gate FAILED:
- PR-AUC 0.3399 < 0.5
- no-show recall 0.8025 < 0.9
```

ภาพที่ 18 ทดลองตั้งเกณฑ์สูงขึ้น (PR-AUC ≥ 0.5 , recall ≥ 0.9) โมเดลไม่ผ่าน gate

```
(base) C:\Users\LENOV\OneDrive\Documents\GitHub\clinic-noshow-prediction>uv run pytest tests/test_ci_model_gate.py -q
.....
5 passed in 0.10s [100%]
```

ภาพที่ 19 test ของ model gate ผ่าน 5 tests

```
Model: exp2_lightgbm_balanced (from model_results.json)
PR-AUC: 0.33988212256344186 (gate >= 0.56)
Recall: 0.8024806201550387 (gate >= 0.9)
Model gate FAILED:
- PR-AUC 0.3399 < 0.56
- no-show recall 0.8025 < 0.9
===== short test summary info =====
FAILED tests/test_ci_model_gate.py::test_committed_model_passes_the_gate - AssertionError: assert ['PR-AUC 0.33...0.8025
< 0.9'] == []
FAILED tests/test_ci_model_gate.py::test_each_threshold_blocks_a_weak_model[val_pr_auc=0.25-PR-AUC] - AssertionError: as
sert 2 == 1
FAILED tests/test_ci_model_gate.py::test_each_threshold_blocks_a_weak_model[val_noshow_recall=0.59-recall] - AssertionEr
ror: assert 2 == 1
FAILED tests/test_ci_model_gate.py::test_cli_exit_codes - assert 1 == 0
4 failed, 1 passed in 0.35s
```

ภาพที่ 20 เมื่อเกณฑ์ไม่ถูกต้อง test ของ model gate fail ตามคาด

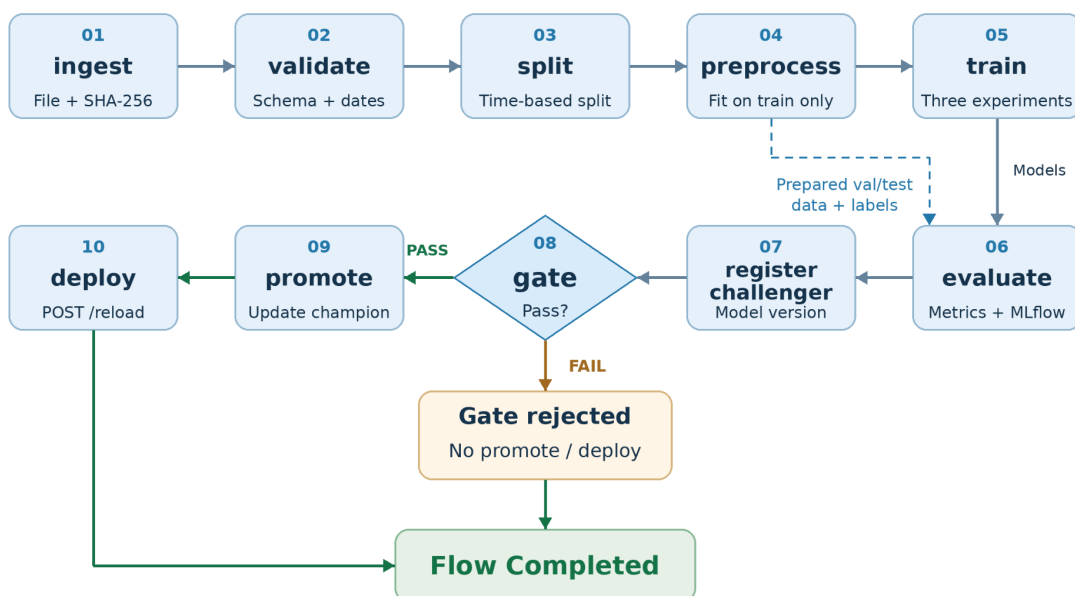
8. Pipeline การทำซ้ำ และการกำกับดูแล (ส่วนที่ 7)

8.1 Pipeline แบบ DAG (Prefect)

`src/noshow/pipeline/flow.py` สั่งรันทั้งกระบวนการด้วยคำสั่งเดียว `make pipeline` (หรือ `uv run python -m noshow.pipeline.flow`)

ingest → validate → split → preprocess → train → evaluate
→ register challenger → gate → promote → deploy (/reload)

- Validation ไม่ผ่าน: หยุด flow, สร้าง Prefect Markdown Artifact และแจ้งเตือน Discord ผ่าน failure hook
- Gate ไม่ผ่าน: flow จบโดยไม่ promote และไม่ deploy
- Gate ผ่าน: promote เป็น champion แล้วเรียก `/reload` และตรวจว่า API โหลด version ตรงกัน



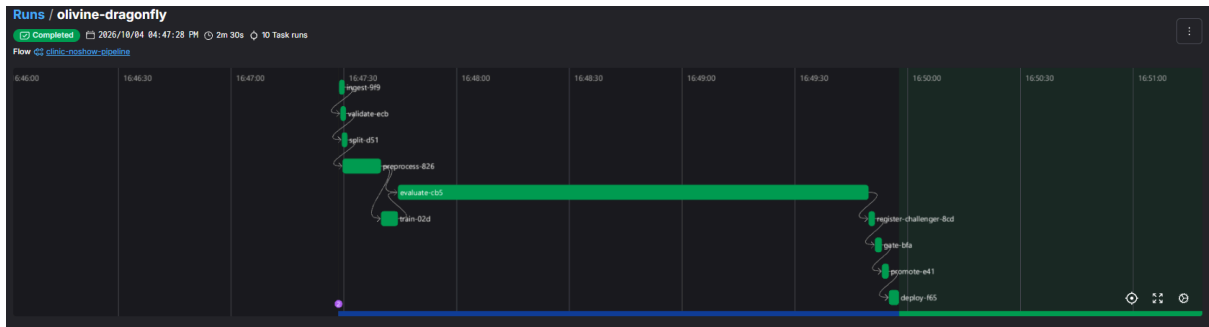
ภาพที่ 8.1 Diagram ลำดับงานและข้อมูลเพิ่มเติมที่ส่งจาก preprocess ไป evaluate

ingest ตรวจไฟล์และ SHA-256; validate ตรวจ schema และวันที่; split ทำความสะอาดและแบ่งตามเวลา พร้อมตรวจว่าแต่ละชุดไม่ว่างและมี target ทั้งสองคลาส; preprocess fit ด้วยข้อมูล train แล้วใช้ตัวแปลงเดียวกันกับ validation และ test

ข้อมูล prepared จาก preprocess ส่งไปทั้ง train และ evaluate โดย train ใช้ข้อมูลฝึก ส่วน evaluate รับโมเดลที่ฝึกแล้วร่วมกับข้อมูล validation/test และ labels เส้นประในภาพจึงแสดง preprocess → evaluate เพิ่มจากลำดับงานหลัก

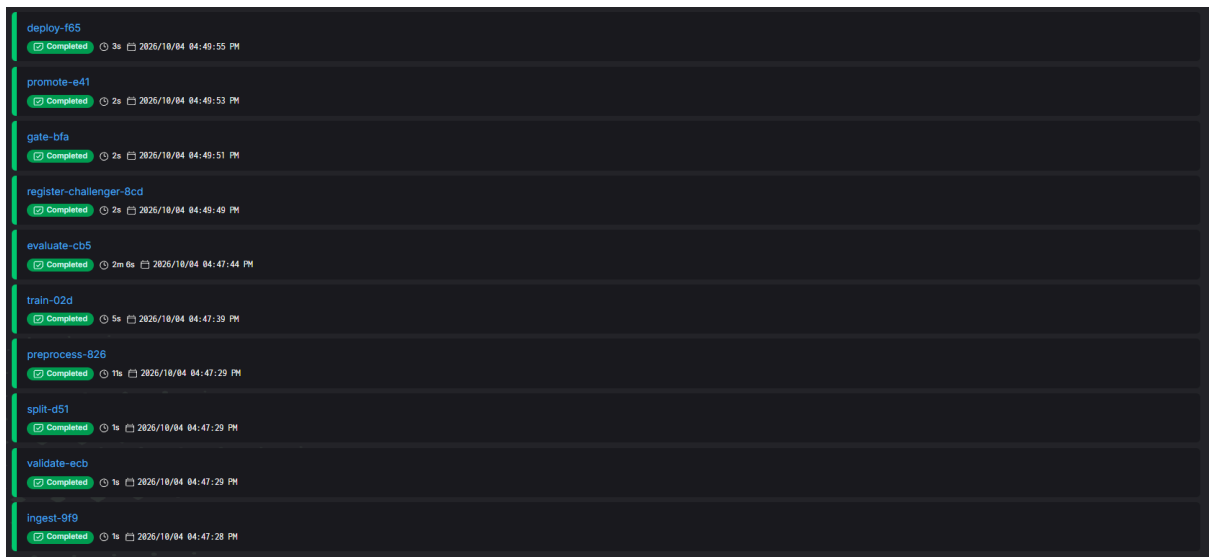
Gate ผ่าน: promote แล้ว deploy เรียก `/reload` พร้อมตรวจเวอร์ชัน API; Gate ไม่ผ่าน: จบ flow โดยไม่ promote และไม่ deploy หากงานเกิด exception จน flow ล้ม จะเรียก failure hook การ reload ล้มไม่ได้ทำให้ rollback อัตโนมัติ ต้องดำเนินการ rollback แยก

8.2 หลักฐานการรัน Pipeline และการแจ้งเตือน



ภาพที่ 8.2 Timeline การรันจริงใน Prefect รอบ olive-dragonfly

ทดสอบจากสำเนาโค้ดใหม่ environment ใหม่ และฐานข้อมูล MLflow ใหม่บนเครื่องเดิม รอบ olive-dragonfly จบในสถานะ Completed ใช้เวลาประมาณ 2 นาที 30 วินาที โมเดลผ่าน Gate และได้รับการ promote เป็น champion Version 1 จากนั้น API โหลดเวอร์ชันดังกล่าวผ่าน /reload สำเร็จ



ภาพที่ 8.3 Task Runs ทั้ง 10 ขั้นตอนแสดงสถานะ Completed

หน้า Task Runs แสดงงานทั้ง 10 ขั้นตอนในสถานะ Completed รวมถึงการลงทะเบียนโมเดล การตรวจ Gate การ promote และการ deploy ผลนี้แสดงว่าการทดสอบจากสำเนาโค้ดใหม่และฐานข้อมูล MLflow ใหม่บนเครื่องเดิมสามารถทำงานต่อเนื่องตั้งแต่ตรวจสอบข้อมูลดิบจนถึงการโหลดโมเดลสำหรับให้บริการได้สำเร็จ



Clinic Pipeline **APP** 02/10/2026 19:30

PIPELINE FAILURE

Flow: validation-failure-demo

Run: phenomenal-crab

Run ID: 39c7ff8a-f1a2-4621-8207-53408658d68d

State: Failed

Reason: Flow run encountered an exception: ValueError: Dataset is empty

ภาพที่ 8.4 การแจ้งเตือน Discord เมื่อทดสอบข้อมูลว่าง

ทดสอบ validation ด้วยข้อมูลว่างให้เกิด ValueError: Dataset is empty จากนั้น failure hook บันทึกชื่อ flow, run, Run ID, สถานะ และสาเหตุใน Prefect Markdown Artifact พร้อมส่งแจ้งเตือน Discord ผ่าน PIPELINE_DISCORD_WEBHOOK_URL การสร้าง Artifact และการส่ง Discord มีการจับข้อผิดพลาดแยกกัน เพื่อให้ยังคงเห็นปัญหาด้านเหตุของ flow

README ของทีมมีวิธีรันผ่าน PowerShell โดยติดตั้ง Git, uv และ Docker Desktop ก่อน จากนั้น clone โปรเจกต์ ติดตั้ง dependencies ตาม uv.lock ดาวน์โหลดข้อมูล ตรวจสอบโค้ด เปิดบริการ และรัน pipeline ตามลำดับ ตัวอย่างคำสั่งหลักมีดังนี้

```
git clone https://github.com/KKU-Noshow-clinic/clinic-noshow-prediction.git
cd clinic-noshow-prediction
uv python install 3.11
uv sync --locked
uv run python scripts/download_data.py
uv run ruff check .
uv run ruff format --check .
uv run pytest -q
$env:API_WORKERS = "2"
$env:API_BATCH = "32"
$env:API_PROFILE = "1"
docker compose up -d --build
$env:PREFECT_API_URL = "http://127.0.0.1:4200/api"
$env:NOSHOW_API_URL = "http://127.0.0.1:8000"
uv run python -m noshow.pipeline.flow
Invoke-RestMethod http://localhost:8000/health | ConvertTo-Json
```

รอให้บริการพร้อมและ /health แสดง model_loaded: true ก่อนทดสอบการทำงาน หากบริการเดิมใช้พอร์ตเดียวกัน ต้องจัดการพอร์ตหรือบริการเดิมก่อนเปิดชุดทดสอบแยก การตั้ง Discord เป็นตัวเลือกเพิ่มเติมโดยตั้ง webhook บนเครื่องที่รัน flow และไม่บันทึก URL กลับลง Git

การตรวจสอบ	ผลที่บันทึกไว้
รันจาก clone, environment และ MLflow ใหม่บนเครื่องเดิม	Pipeline สำเร็จ และ API โหลด champion Version 1
ตรวจสอบ parity โมเดลกับ API	เปรียบเทียบ 300 แถวผ่าน /predict_batch และ 20 แถวผ่าน /predict; alert mismatches = 0
ผลต่างคะแนน parity สูงสุด	batch 5.00e-07 และ single 4.97e-07 ซึ่งอยู่ในระดับการปิดทศนิยม
Ruff และชุดทดสอบที่รวมงานทีมแล้ว ณ รอบตรวจ 4 ตุลาคม 2569	ruff check และ format --check ผ่าน; pytest 105 passed, 4 warnings

การทดสอบจาก clone ใหม่บนเครื่องเดิมแสดงว่ารันได้โดยไม่พึ่งไฟล์ข้อมูลและฐาน MLflow ในโฟลเดอร์ทำงานเดิม แต่ไม่ใช้การติดตั้งบนเครื่องหรือระบบปฏิบัติการที่ไม่เคยมีเครื่องมือนี้อีกก่อน

8.4 Git workflow และการรันจากเครื่องเปล่า

- ห้าม push ตรงเข้า main ทุกงานทำบน branch (feat/<ชื่อ>-<เรื่อง>) แล้วเปิด Pull Request ที่ต้องผ่าน CI และมีผู้ approve อย่างน้อย 1 คน
- ทุกคน commit งานของตัวเองด้วยบัญชีของตัวเอง

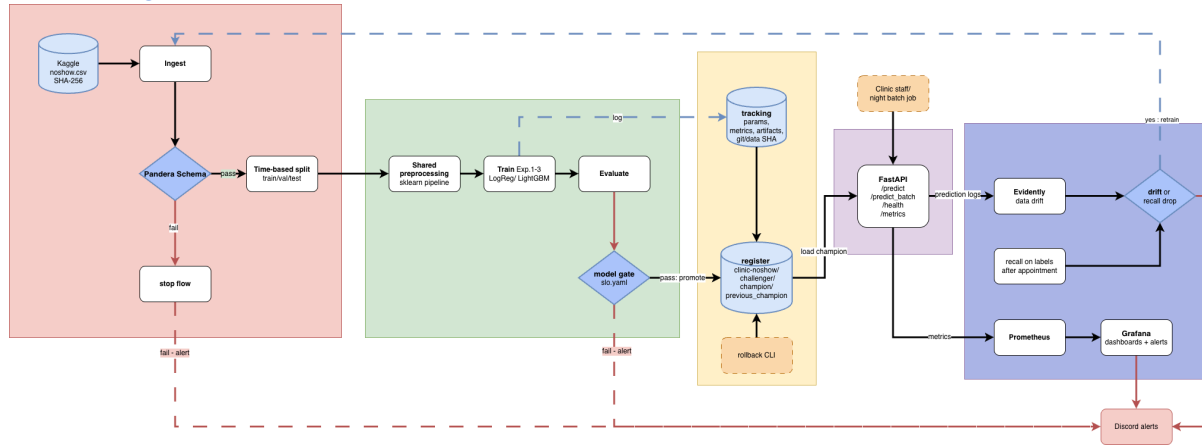
- README มีคำสั่งรันจากเครื่องเปล่า: `make setup` → `make data` → `make test` → `make up` → `make pipeline`
- ล็อกเวอร์ชันไลบรารีทั้งหมดใน `uv.lock` และล็อกเวอร์ชัน image ใน `docker-compose.yml` (เช่น `mlflow:v3.16.1`, `grafana:12.1.0`)

อ้างอิง: [README.md](#)

9. สถาปัตยกรรมและการทดสอบ (ส่วนที่ 8)

9.1 แผนภาพสถาปัตยกรรม

[Link to Diagram \(Click\)](#)



10. การใช้เครื่องมือ AI

ตามข้อกำหนดรายวิชา สมาชิกแต่ละคนระบุส่วนที่ใช้ AI ช่วย และต้องอธิบายโค้ดที่ตนเอง commit ได้ทุกบรรทัด

สมาชิก	ส่วนที่ใช้ AI ช่วย
นายพิรพงษ์ ทองฤทธิ์ (Data & Validation)	ใช้ AI อธิบายขั้นตอนและให้ตัวอย่างโค้ด EDA, clean, split, schema, tests และสคริปต์จำลอง drift พิมพ์โค้ด รันทดสอบ ตรวจสอบผล และตัดสินใจกฎ cleaning เอง และใช้ AI ช่วยเรียบเรียงคำในรายงานฉบับนี้
นายศรัณยู เจริญผล	ตรวจสอบระบบ: ทดสอบรันระบบจากเครื่องเปล่า (make setup → data → test → up → pipeline) และ review โค้ดใน branch ของสมาชิกก่อน merge ทำตารางงานรายวันและแผนภาพสถาปัตยกรรม (Mermaid)
นายธันว์ สว่างศรี (Features & Modeling)	ใช้ AI ช่วยอธิบาย แนะนำ เขียน/ปรับปรุง ตรวจสอบ การสร้าง features, preprocessing, การฝึกและประเมินโมเดล, tests และช่วยเรียบเรียงสไลด์นำเสนอ
นายกิตติพันธ์ ไชไพบรวัน	ใช้ ChatGPT/Codex ช่วยเขียนและปรับโค้ด Prefect pipeline, MLflow registry, Gate, promote, rollback, การเรียก API reload, Discord failure notification, tests และ script สำเร็จ
นายภูรินทร์ ศรีฐาน	ใช้ Claude ช่วยเขียนโค้ด serving (FastAPI, micro-batching, การโหลดโมเดลจาก registry), Dockerfile/docker-compose, สคริปต์ load test/check_slo/parity/profiling, ชุดทดสอบ serving และ docs/serving.md
นายทีฆานนท์ อัสวะภูมิ	ใช้ ChatGPT/Codex ช่วยอธิบายและปรับโค้ดระบบ Monitoring ได้แก่ Prometheus, Grafana, Evidently, Metrics Exporter, การตรวจ Data/Concept Drift และ Monitoring Policy รวมถึงช่วยวิเคราะห์ Error และแนะนำการตั้งค่า Docker/Monitoring
นายณัฏทพงศ์ พยัคมะเริง	ใช้ Claude ช่วยปรับโค้ด .github/workflows/ci.yml, data_validation.py, model_gate.py และ tests/test_ci_*.py รวมถึงช่วยเรียบเรียง docs/ci.md โดยกำหนดเองว่า CI ต้องตรวจสอบอะไรตามหลัก Code, Data, Model เลือกเคสข้อมูลเสียและใช้เกณฑ์ gate ตาม slo.yaml ของทีม รันทดสอบ ตรวจสอบผล และทดลองป้อนข้อมูลเสียกับโมเดลที่แยบน GitHub Actions เพื่อยืนยันว่า CI บล็อกได้จริง

แหล่งอ้างอิง

GitHub repository: <https://github.com/KKU-Noshow-clinic/clinic-noshow-prediction>

ชุดข้อมูล: [Kaggle – Medical Appointment No Shows](#)